
Genetic Algorithm

1. GA implementation

Correct implementation of all parts of the GA algorithm.

2. GA results & plots

The plots should demonstrate proper searching and convergence behavior.

- A. Plot the average and the highest fitness score of the population versus the generation of the population.
- B. Select the vector with the highest fitness score in the last generation of the population. Report the hyperparameters.
- C. Train the model on the training and validation data using the selected hyperparameters. Plot the training F1 score versus the training epochs.

D. Evaluate the model on the test data. Report the test F1 score.

In what follows the following abbreviations refer to:

- E → Number of Epochs
- M → mutation probability
- S → number of new children in each generation which is also equal to the number
- P → population size
- I → max number of iterations
- Lr → learning rate

Note that some codes were interrupted because of service. Some code reached their maximum iterations. And some code had a preset convergence criteria that wasn't optimal, so it led to early stopping.

However, my main convergence criteria are the max iterations or the condition that avg_fitness becomes more than 0.95 or the max_fitness becomes more than 0.99. Different convergence criteria can be set.

Please note that initially I tested one model, and realized that lr=0.01 is the best value for the learning rate, so I tested most runs with it. Regarding the E value, about 500-600 is best, however it takes so much computation time, so I tested with fewer E values.

i. Run A

E= 100, M=0.2, I=10,000, S=14, P=30, lr=0.01

```
bestoflast = history_best_dict[9999]
```

```
[(23, 'relu')]
```

```
fitness_dict[history_best_dict[9999][0]]
```

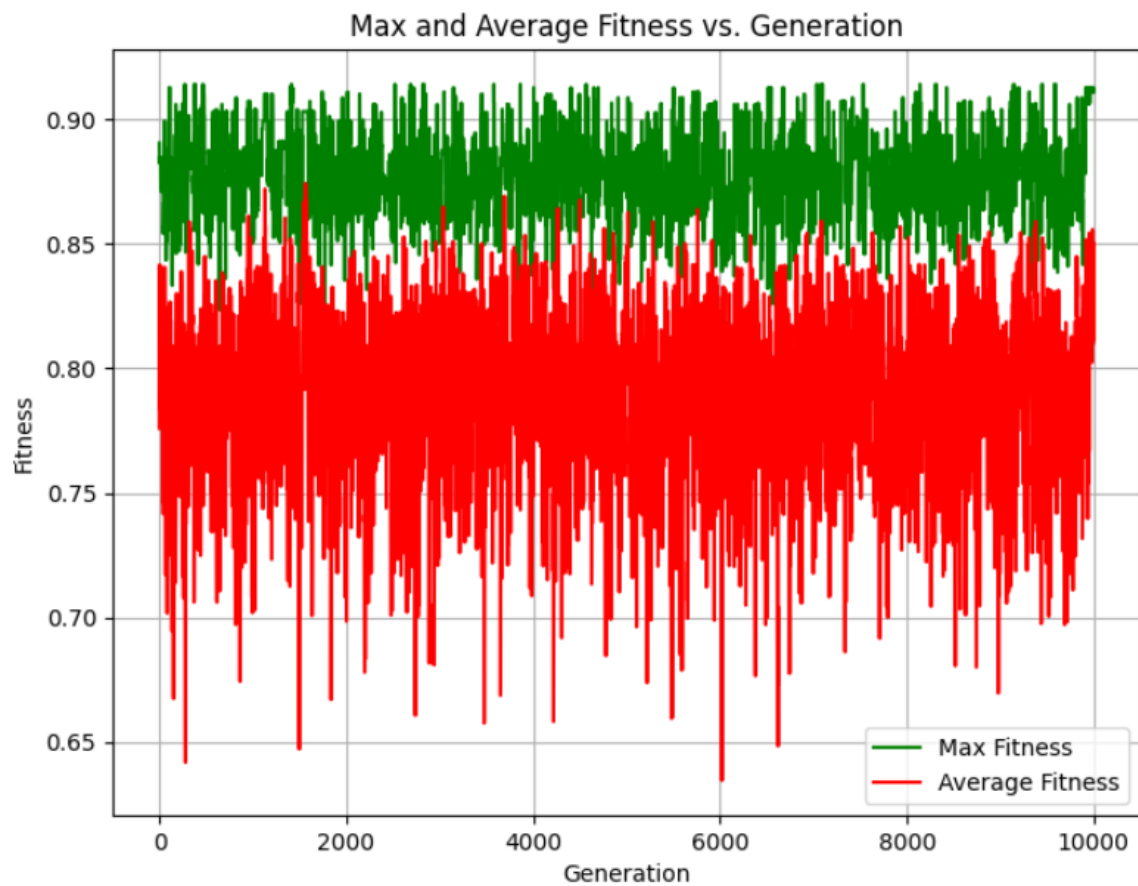
```
0.9110402464866638
```

```
key_with_max_value = max(fitness_dict, key=fitness_dict.get)
print(key_with_max_value)
max_fitness_of_all_time = fitness_dict[key_with_max_value]
print(max_fitness_of_all_time)
```

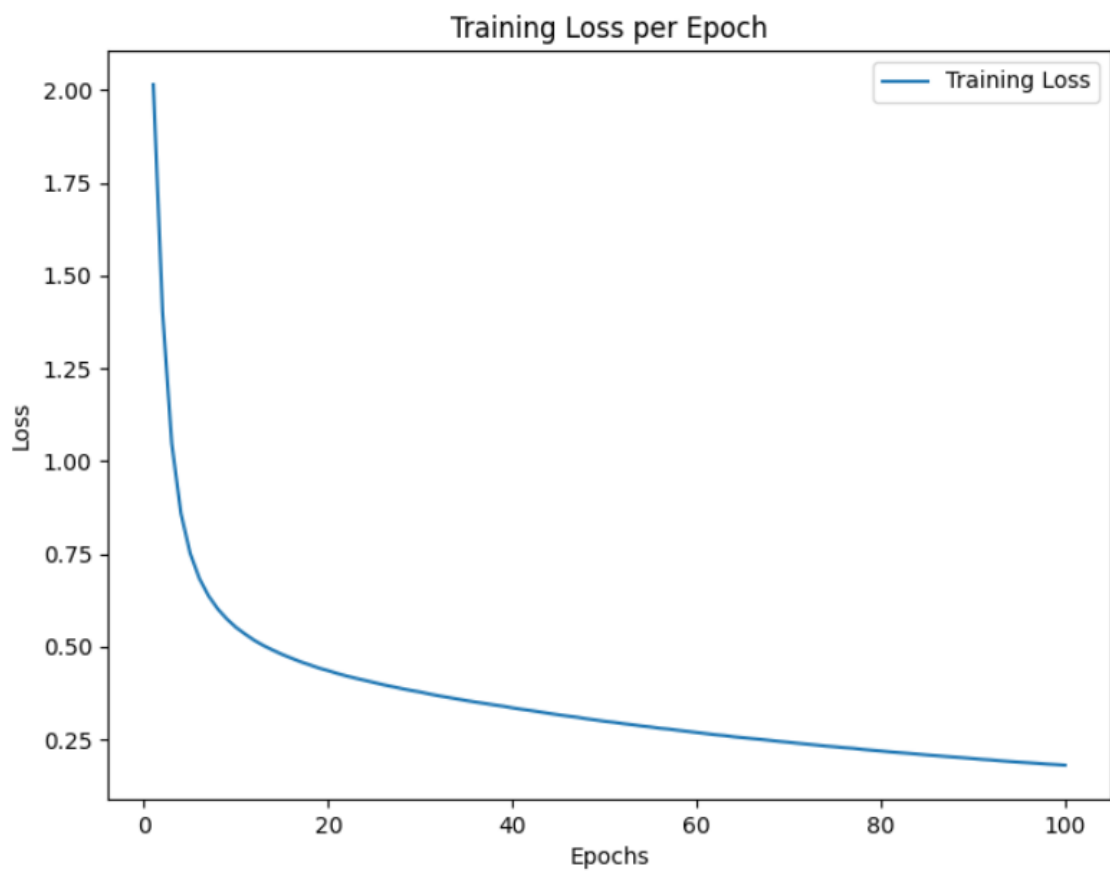
```
(16, 'tanh')
```

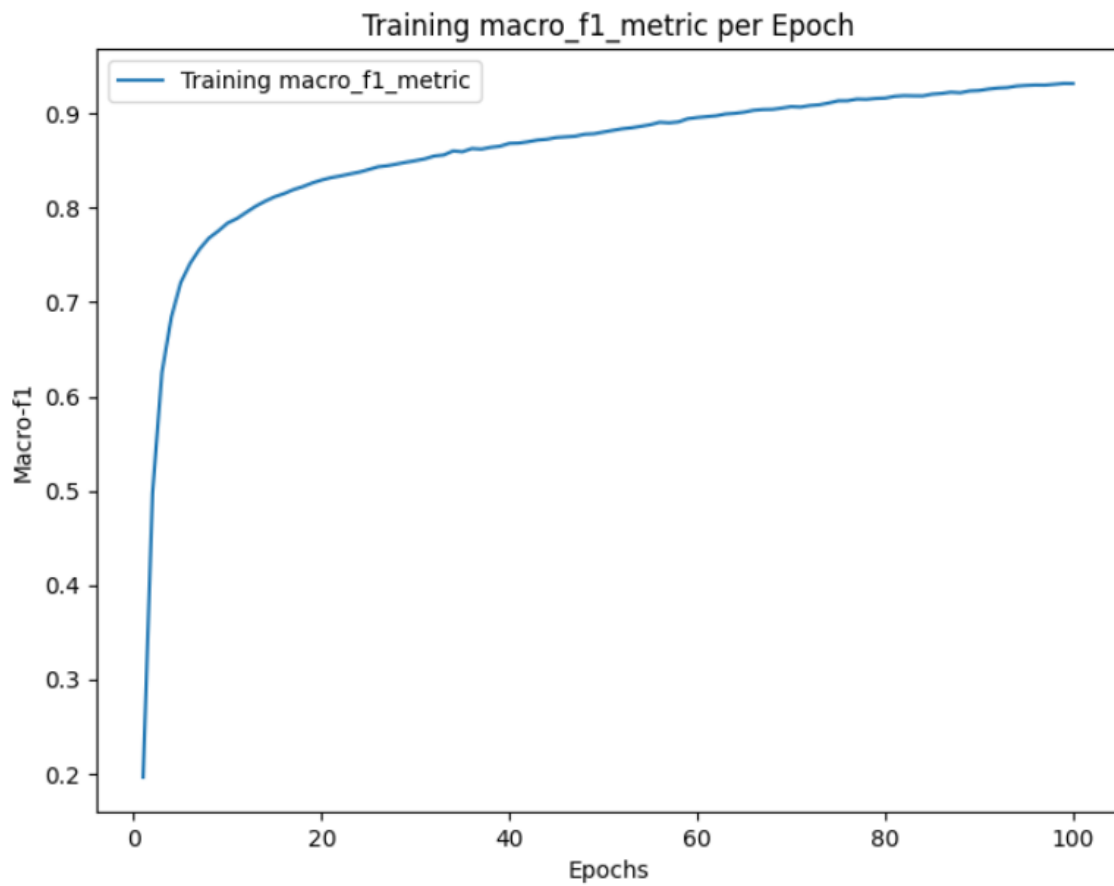
```
0.9140990972518921
```

- Best hyperparameters of the last generation = (23, 'relu')

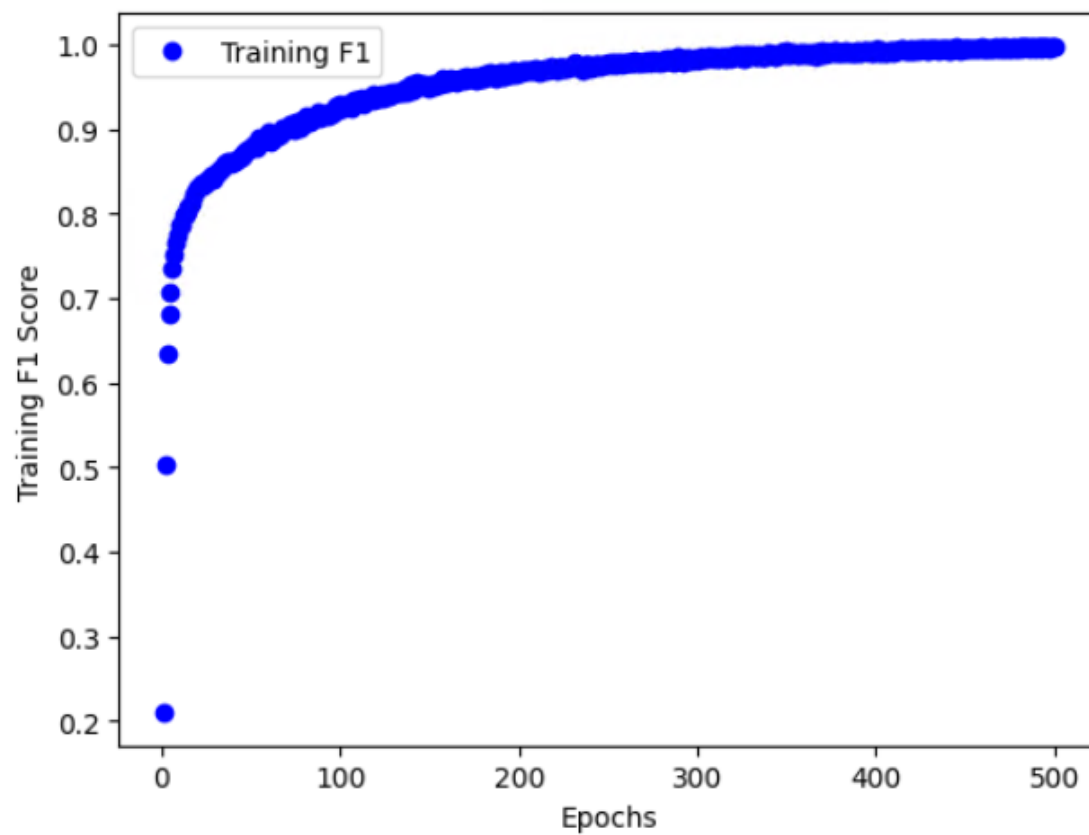


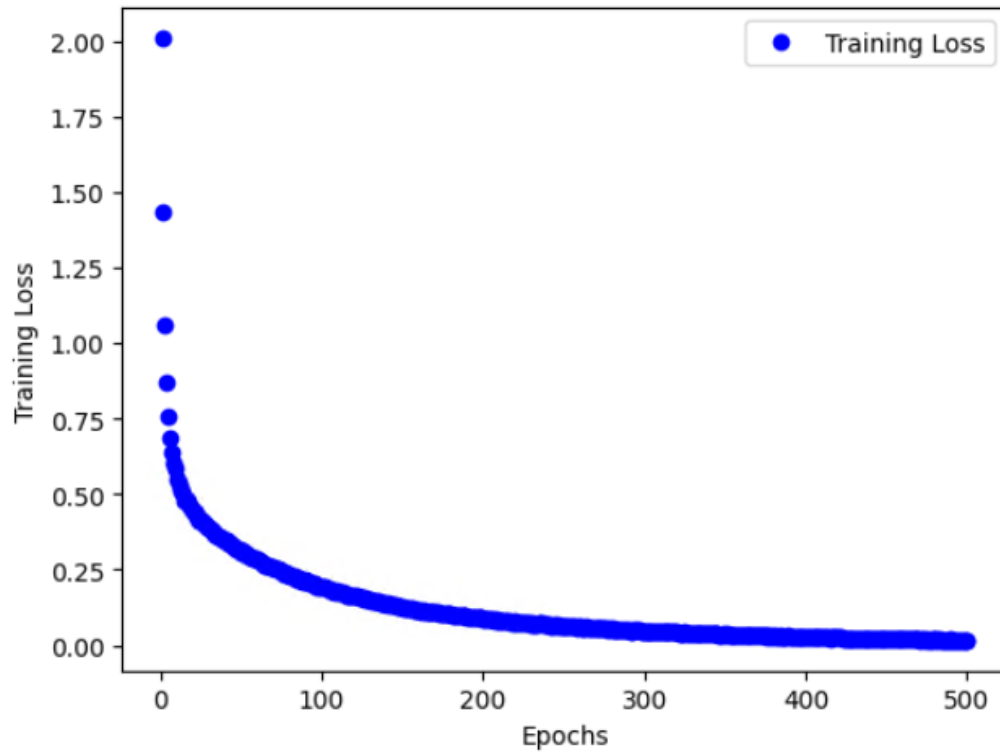
The plots below are of the training on the model using (23, 'relu'), with $E=100$, $lr=0.01$.





The plots below are of the training on the model using (23, 'relu'), with $E=500$, $lr=0.01$.
(Note that I tested $lr=0.001$, 0.0001 , and 0.01 , and found out that $lr=0.01$ is best).

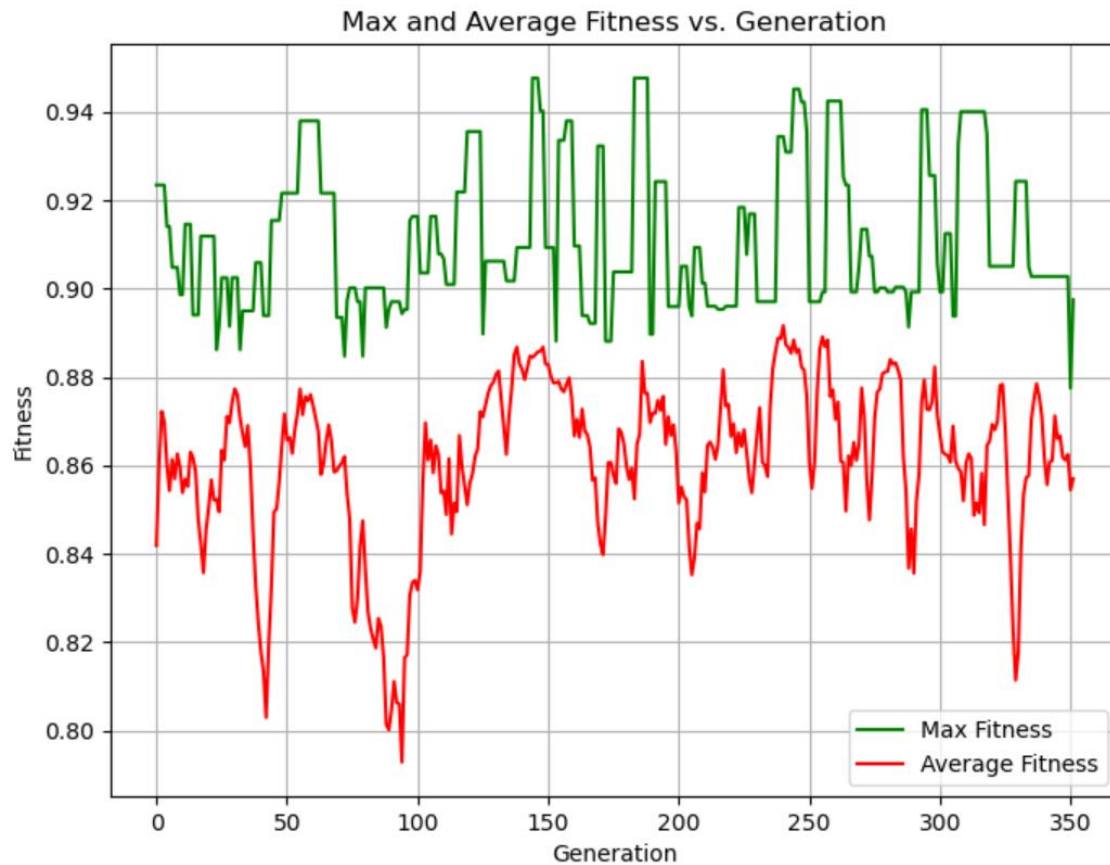




```
33/33 [=====] - 0s 1ms/step
Test F1 Score: 0.963251310560377
33/33 [=====] - 0s 5ms/step - loss: 0.1496 - macro_f1: 0.9557
Test Loss: 0.14956676959991455
```

ii. Run B

E=300, I=1000, M=0.15, S=10, P=30, lr=0.01



```
: bestoflast = history_best_dict[int(len(history_best_dict)-1)]
print(bestoflast)
```

```
[(266, 'relu')]
```

```
: fitness_dict[bestoflast[0]]
```

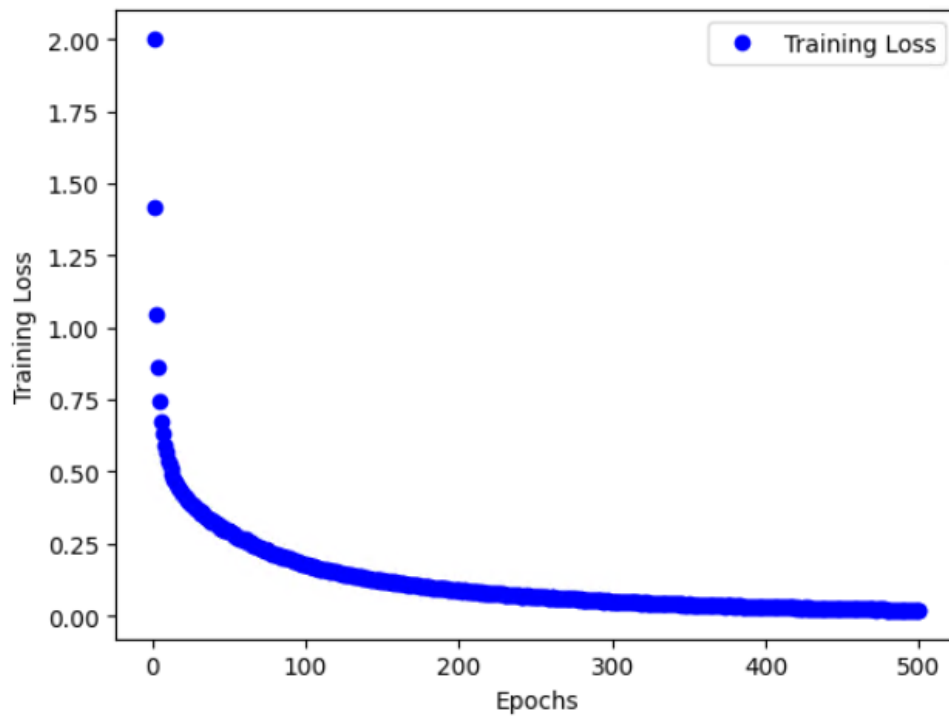
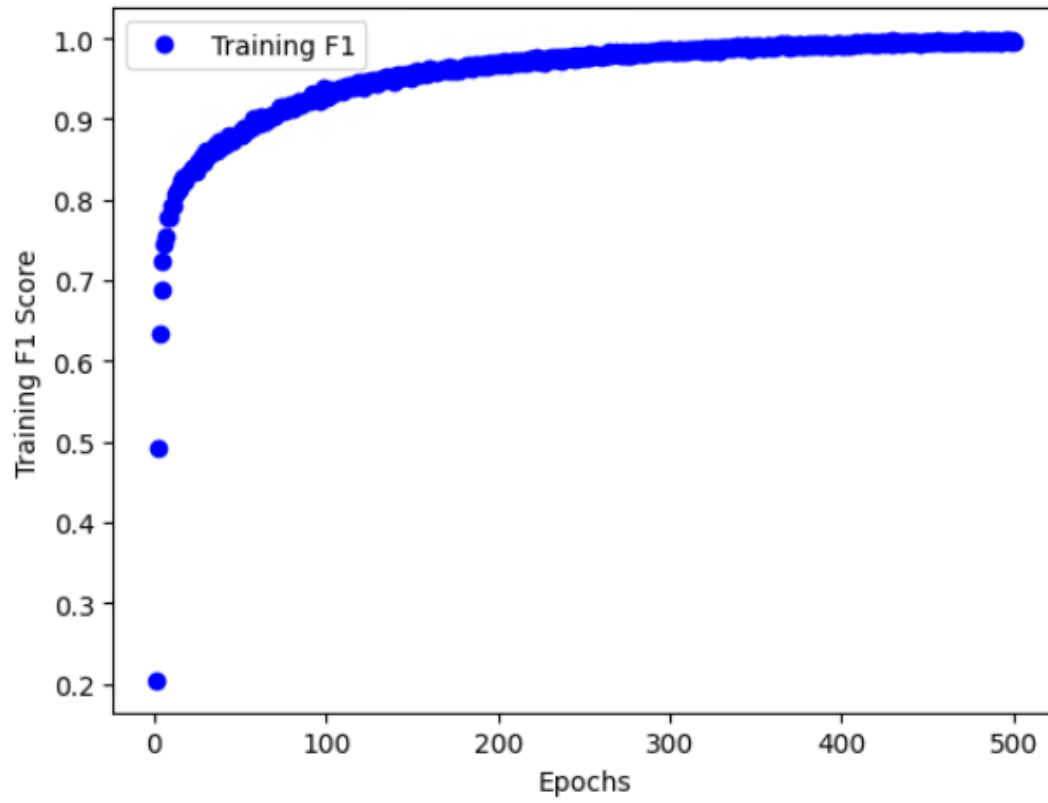
```
: 0.8975375890731812
```

```
: key_with_max_value = max(fitness_dict, key=fitness_dict.get)
print(key_with_max_value)
max_fitness_of_all_time = fitness_dict[key_with_max_value]
print(max_fitness_of_all_time)
```

```
(22, 'tanh')
```

```
0.9477418661117554
```

Since the model didn't converge, instead of looking at the last best chromosome of the last generation, which is (266,'relu'), we will look at (22, tanh).



```
33/33 [=====] - 0s 1ms/step  
Test F1 Score: 0.9694376968141395  
33/33 [=====] - 0s 7ms/step - loss: 0.1329 - macro_f1: 0.9644  
Test Loss: 0.13289687037467957
```

(Tested with E=500, and lr=0.01)

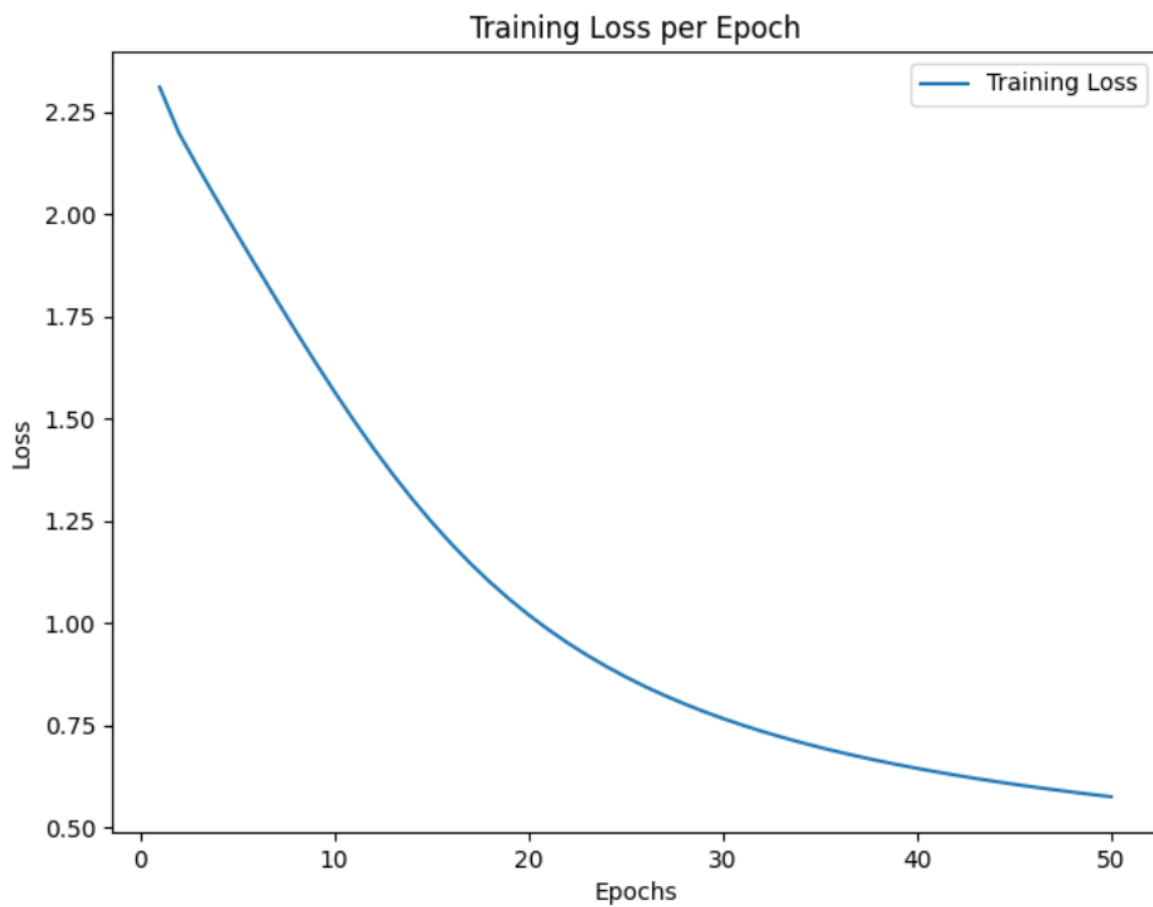
iii. Run C

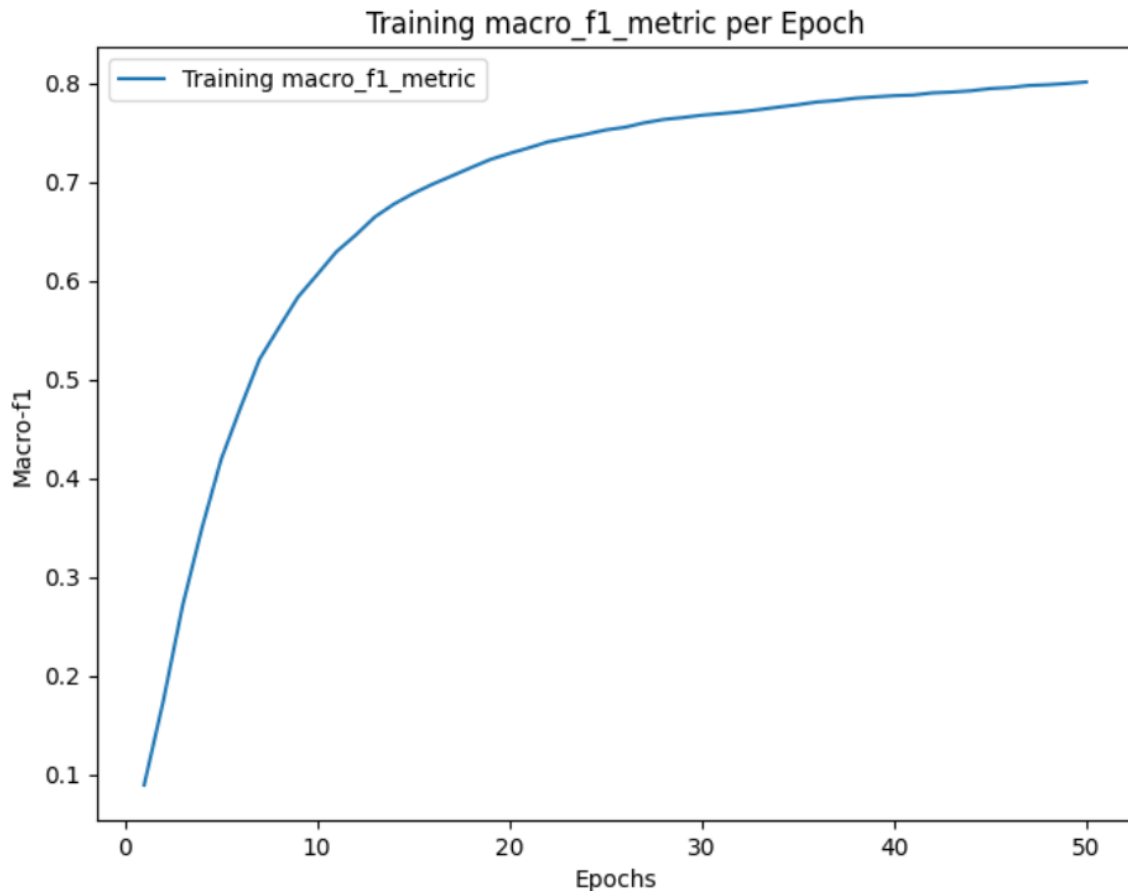
E=50, 500 generations, m=0.24, S=20, P=40, lr=0.001

- Best hyperparameters of the last generation = 23, 'relu'
- Combined-training-macro-f1 = 0.8002
- Test-macro-f1=0.799, test-loss=0.5559



The plots below when tested with E=50, and lr=0.001.

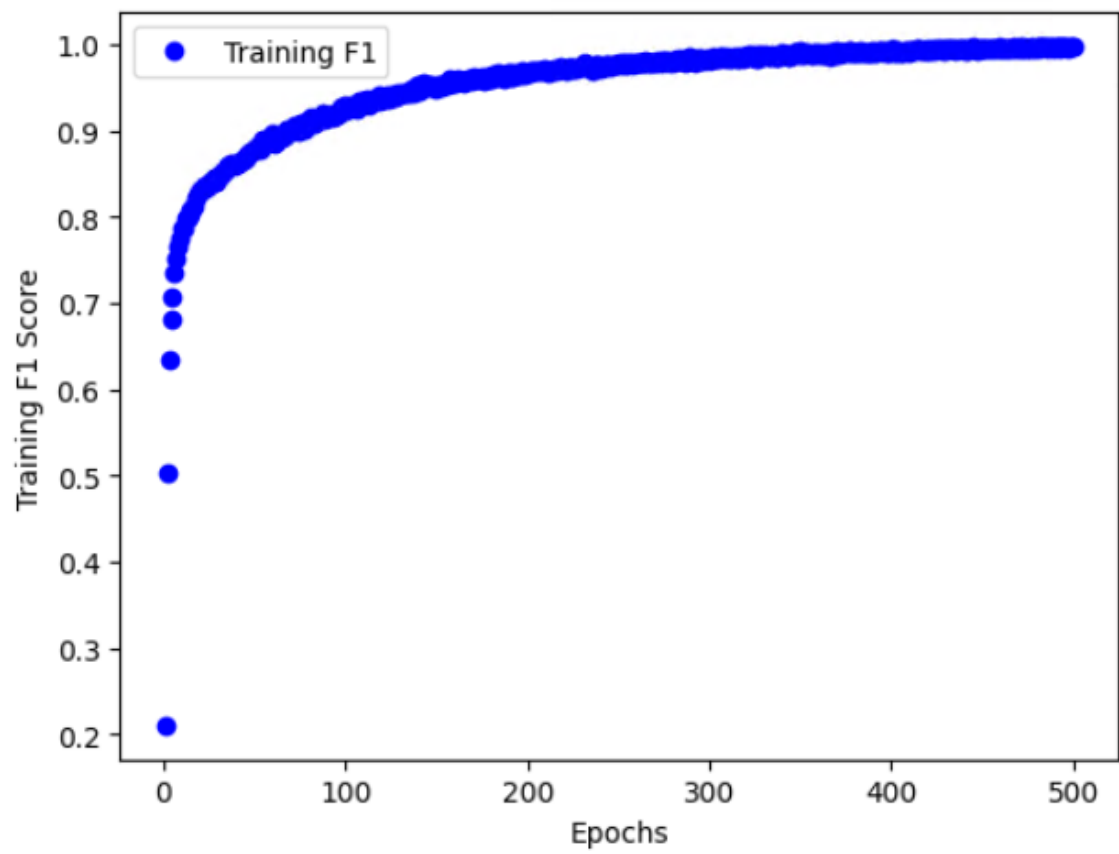


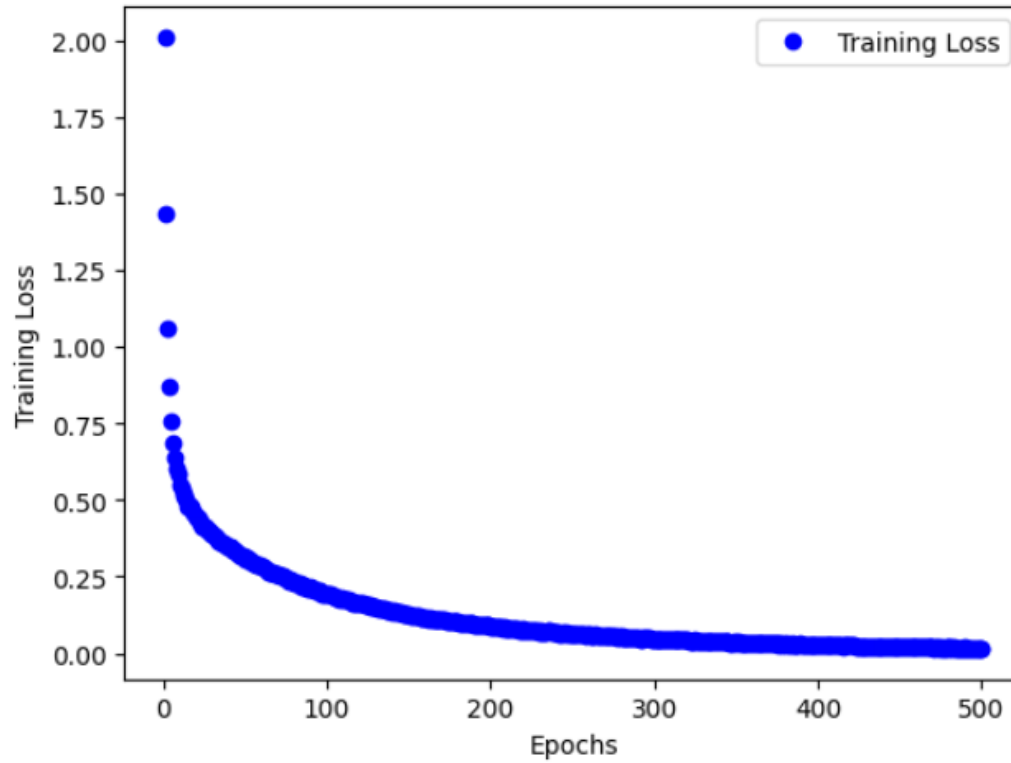


```
best chromosome of this generation: [(23, 'relu')]
max fitness: 0.795830488204956 avg fitness: 0.5369256468489766
Epoch 1/50
```

```
Combined Train f1: 0.8002
 1/46 [.....] - ETA: 0s - loss: 0.7977 - macro_f1: 0.73
15/46 [=====>.....] - ETA: 0s - loss: 0.5689 - macro_f1: 0.79
28/46 [=====>.....] - ETA: 0s - loss: 0.5678 - macro_f1: 0.78
42/46 [=====>....] - ETA: 0s - loss: 0.5759 - macro_f1: 0.78
46/46 [=====>] - 0s 4ms/step - loss: 0.5559 - macro_f1:
0.7990
Test F1: 0.7990
Test Loss: 0.5559
```

However, if we increase E=500 and lr to 0.01:





33/33 [=====] - 0s 1ms/step

Test F1 Score: 0.963251310560377

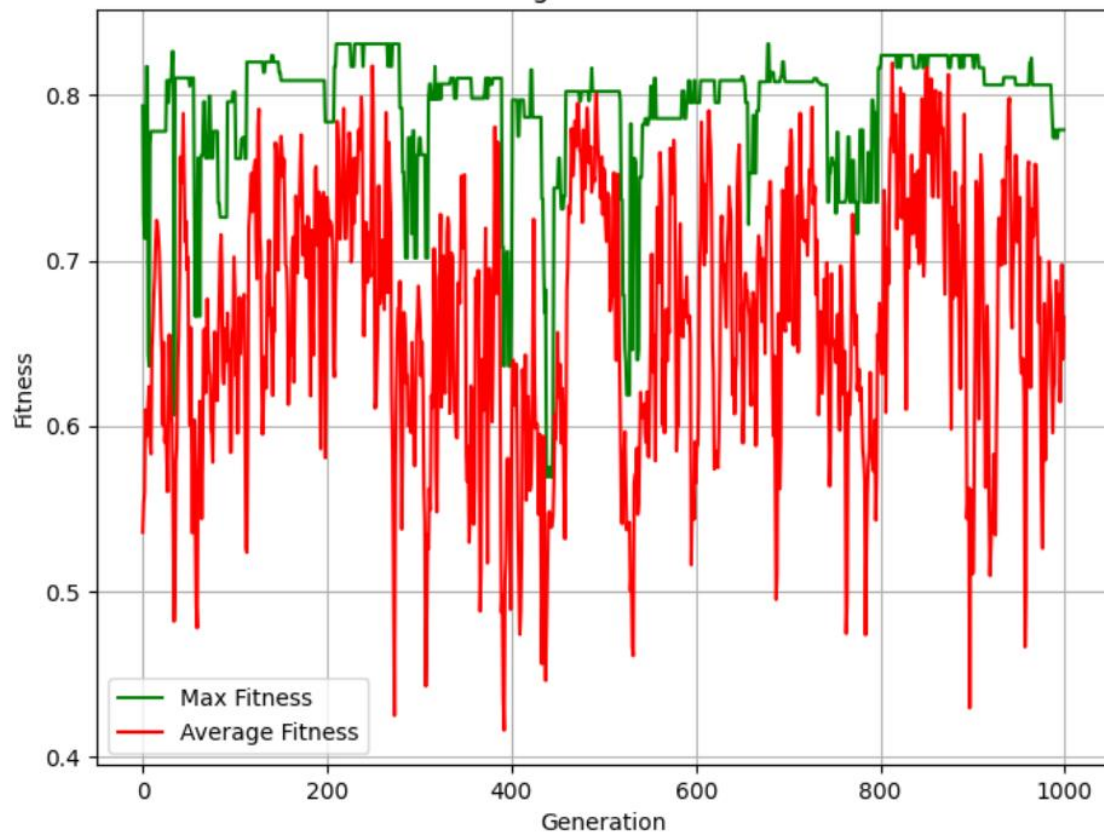
33/33 [=====] - 0s 5ms/step - loss: 0.1496 - macro_f1: 0.9557

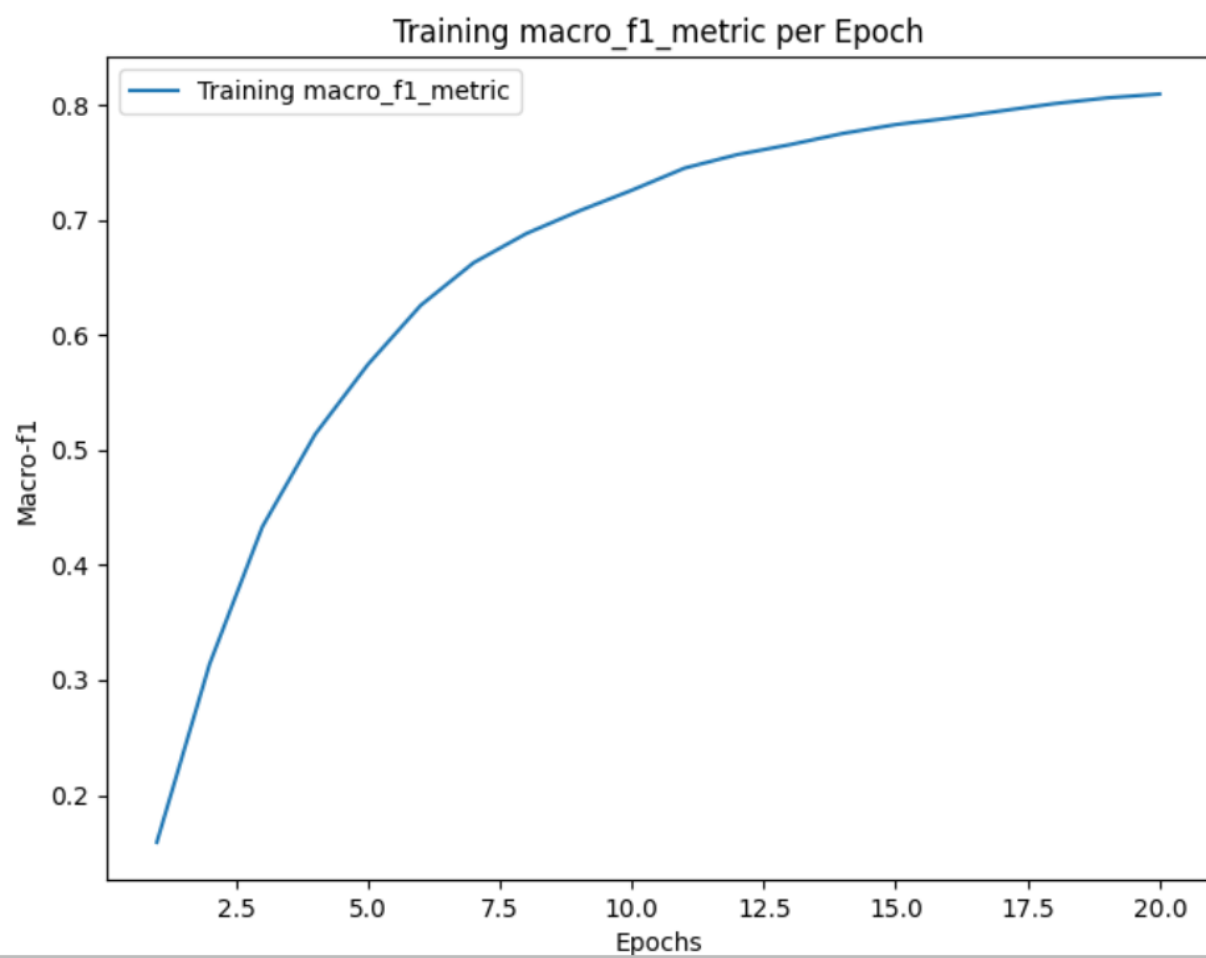
Test Loss: 0.14956676959991455

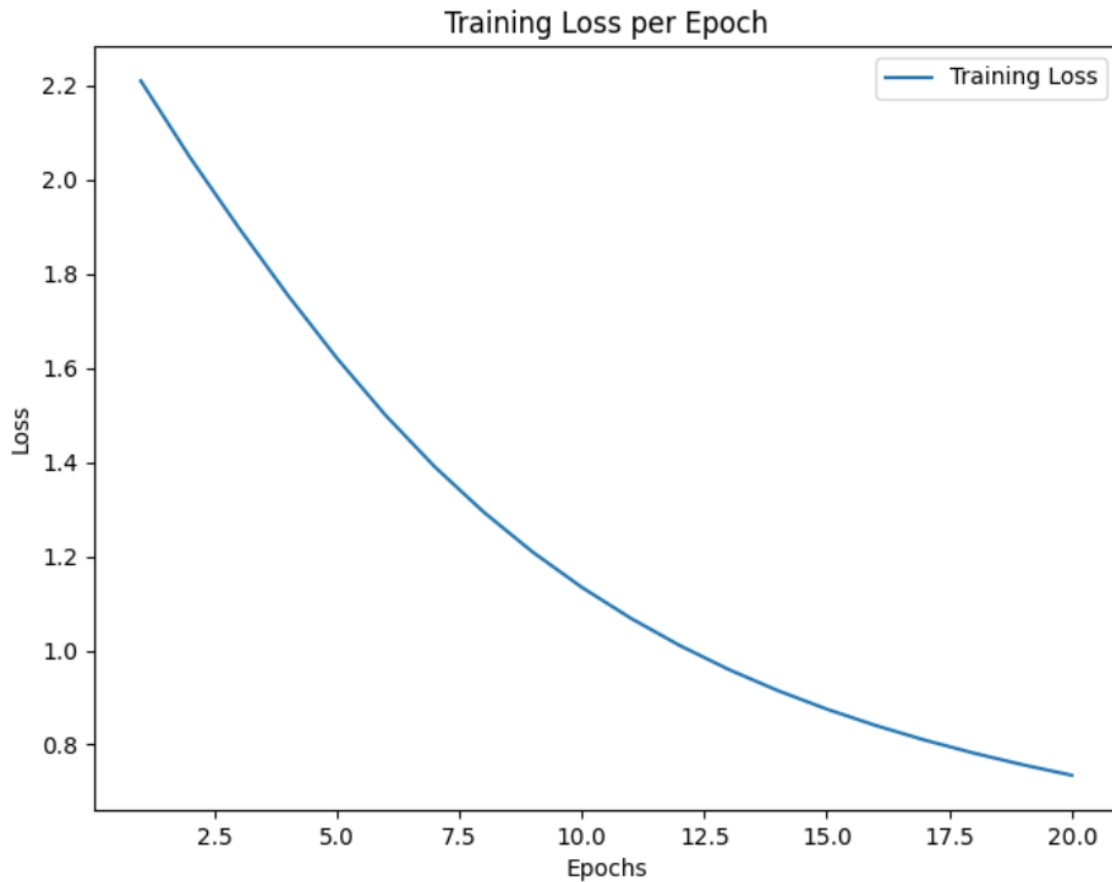
iv. Run D

E=20, 1000 generations, m=0.15, S=8, P=15, lr=0.01

Max and Average Fitness vs. Generation







- Best hyperparameters of the last generation = 134, 'tanh'
- Training the model with the best hyperparameters: train_f1 = 0.7394, train_loss=0.738
- Test-f1 = 0.8277, test-loss = 0.8277

```
generation: 999
best chromosome of this generation: [(134, 'tanh')]
max fitness: 0.779135525226593 avg fitness: 0.6659666314721108
Epoch 1/20
1/72 [.....] - ETA: 19s - loss: 2.7695 - macro_f1: 0.0
```

```
Epoch 20/20
1/72 [.....] - ETA: 0s - loss: 0.8316 - macro_f1: 0.79
13/72 [====>.....] - ETA: 0s - loss: 0.7652 - macro_f1: 0.78
25/72 [=====>.....] - ETA: 0s - loss: 0.7441 - macro_f1: 0.80
36/72 [======>.....] - ETA: 0s - loss: 0.7420 - macro_f1: 0.80
48/72 [======>.....] - ETA: 0s - loss: 0.7421 - macro_f1: 0.80
61/72 [======>.....] - ETA: 0s - loss: 0.7420 - macro_f1: 0.80
72/72 [=====] - 0s 4ms/step - loss: 0.7349 - macro_f1: 0.8095
Combined Train f1: 0.7394
8/8 [=====] - 0s 4ms/step - loss: 0.8277 - macro_f1: 0.7380
Test F1: 0.7380
Test Loss: 0.8277
```

However, there were better results in previous generations:

```
generation: 849
best chromosome of this generation: [(28, 'relu')]
max fitness: 0.8241276741027832 avg fitness: 0.8130544026692709
generation: 850
best chromosome of this generation: [(28, 'tanh')]
max fitness: 0.8166289329528809 avg fitness: 0.8166289329528809
generation: 851
best chromosome of this generation: [(28, 'tanh')]
max fitness: 0.8166289329528809 avg fitness: 0.8166289329528809
```

```
best chromosome of this generation: [(23, 'tanh')]
max fitness: 0.8310635089874268 avg fitness: 0.7778128604094188
generation: 235
best chromosome of this generation: [(23, 'tanh')]
max fitness: 0.8310635089874268 avg fitness: 0.7794652918974558
generation: 236
```

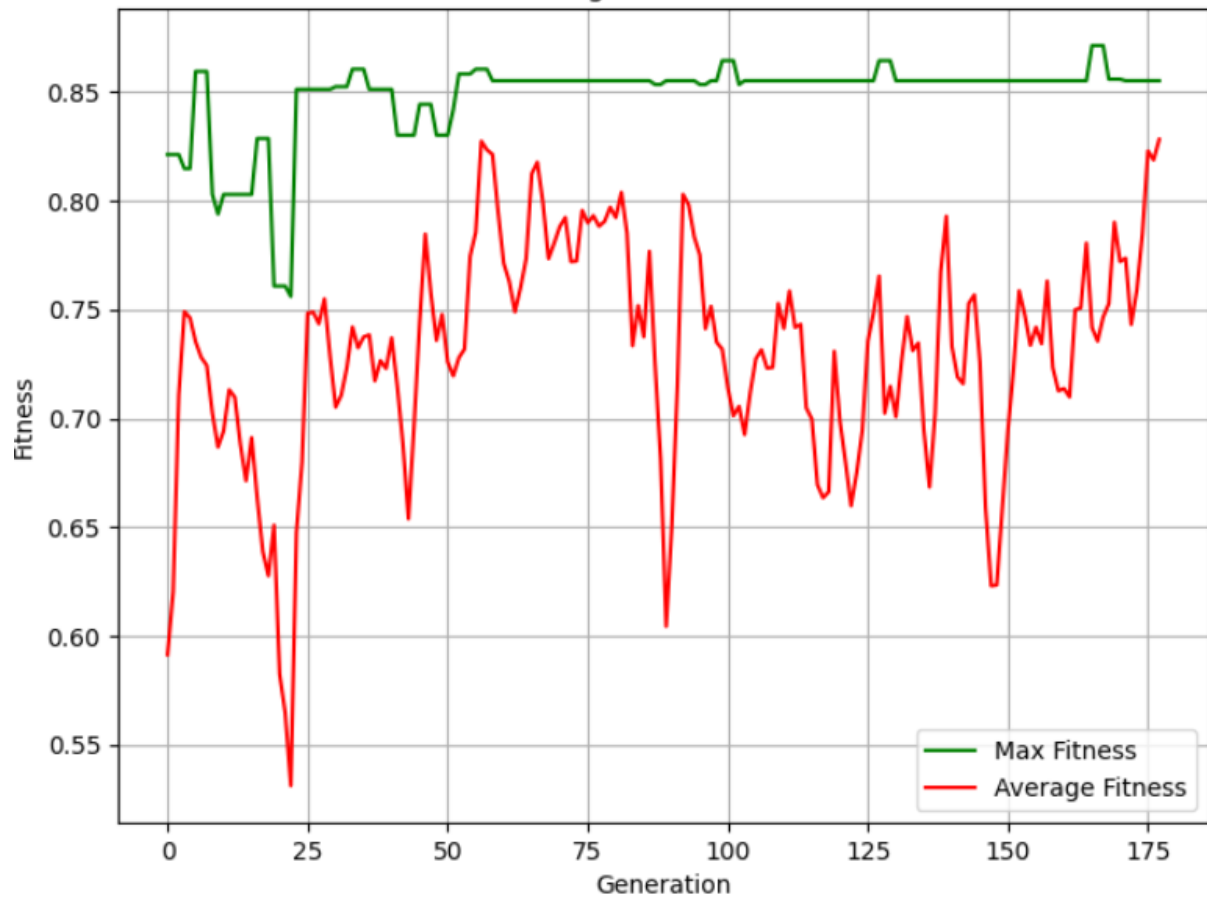
Although, (“tanh”, 134) is the best chromosome of the last generation. It doesn’t seem to be good enough. So, no need to test it further on E=500.

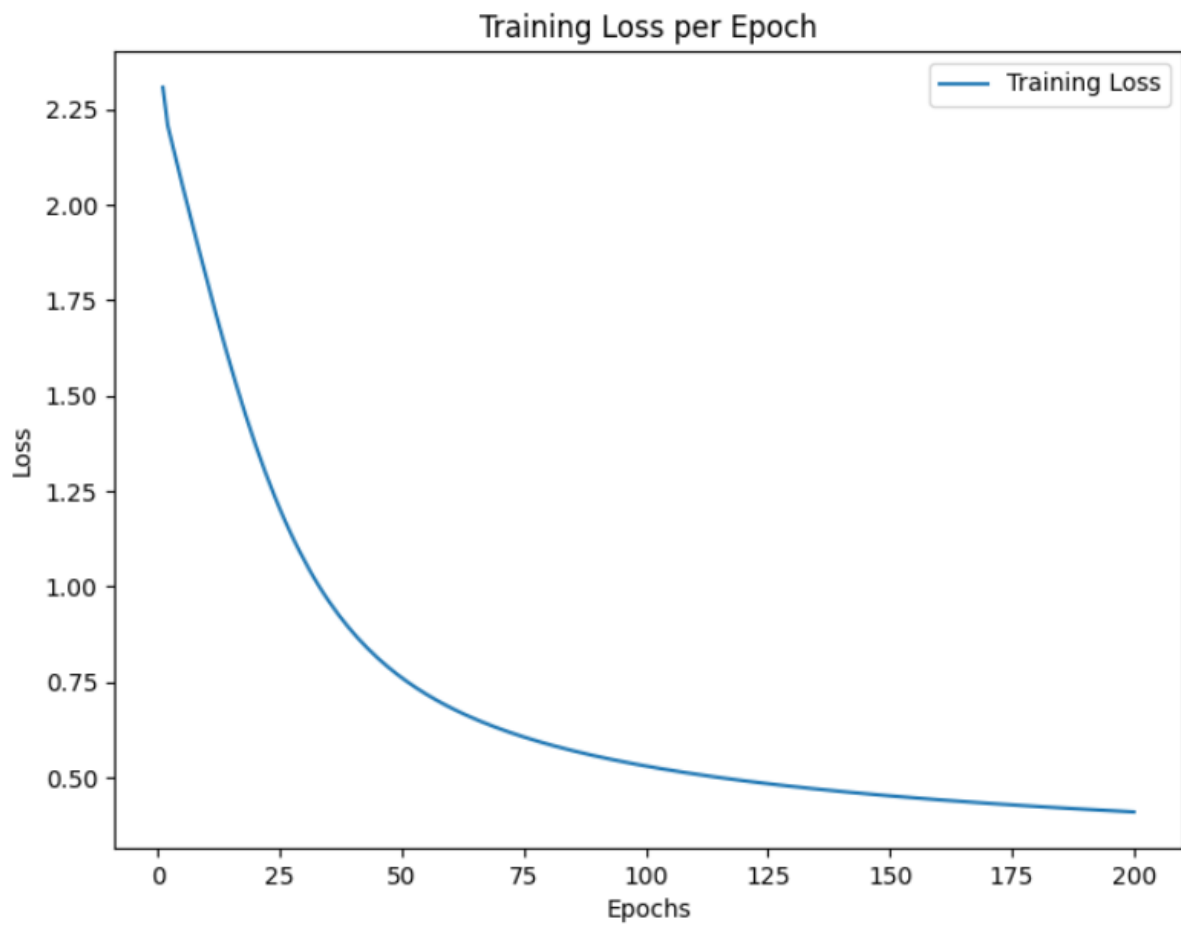
v. Run E

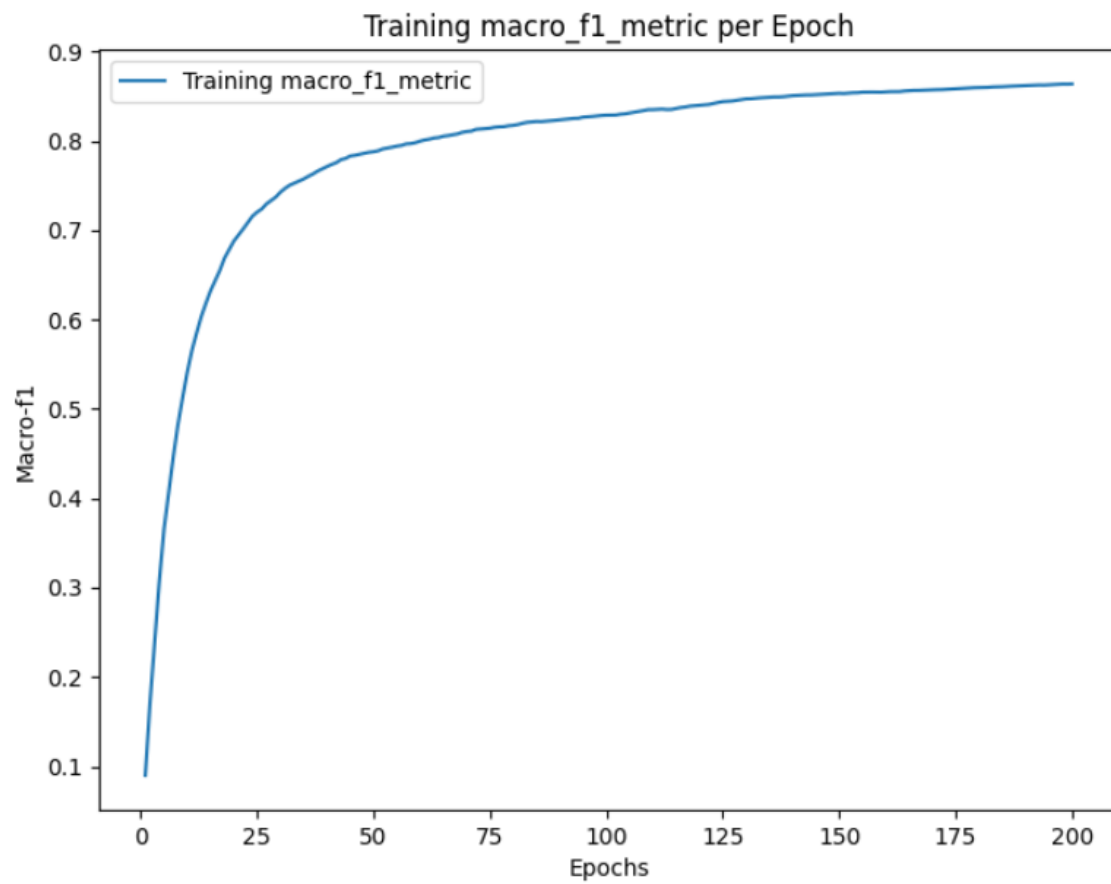
E=200, I=1000, M=0.15, S=8, P=24, lr=0.001

I added a convergence criterion that wasn’t proper so the algorithm would stop when avg_fitness and highest_fitness are within 0.03.

Max and Average Fitness vs. Generation







```
245/245 [=====] - 1s 4ms/step - loss:
: 0.8637
Combined Train f1: 0.8643
1/27 [>.....] - ETA: 0s - loss: 0.4047
14/27 [=====>.....] - ETA: 0s - loss: 0.4201
27/27 [=====] - ETA: 0s - loss: 0.4032
27/27 [=====] - 0s 4ms/step - loss: 0.
0.8483
Test F1: 0.8483
Test Loss: 0.4032
```

```
[3]: bestoflast = history_best_dict[int(len(history_best_dict)-1)]
      print(bestoflast)

      [(39, 'relu')]
```

```
[4]: fitness_dict[bestoflast[0]]
```

```
[4]: 0.8553190231323242
```

```
[5]: key_with_max_value = max(fitness_dict, key=fitness_dict.get)
      print(key_with_max_value)
      max_fitness_of_all_time = fitness_dict[key_with_max_value]
      print(max_fitness_of_all_time)

      (19, 'tanh')
      0.8714563846588135
```

```
1. 1.
```

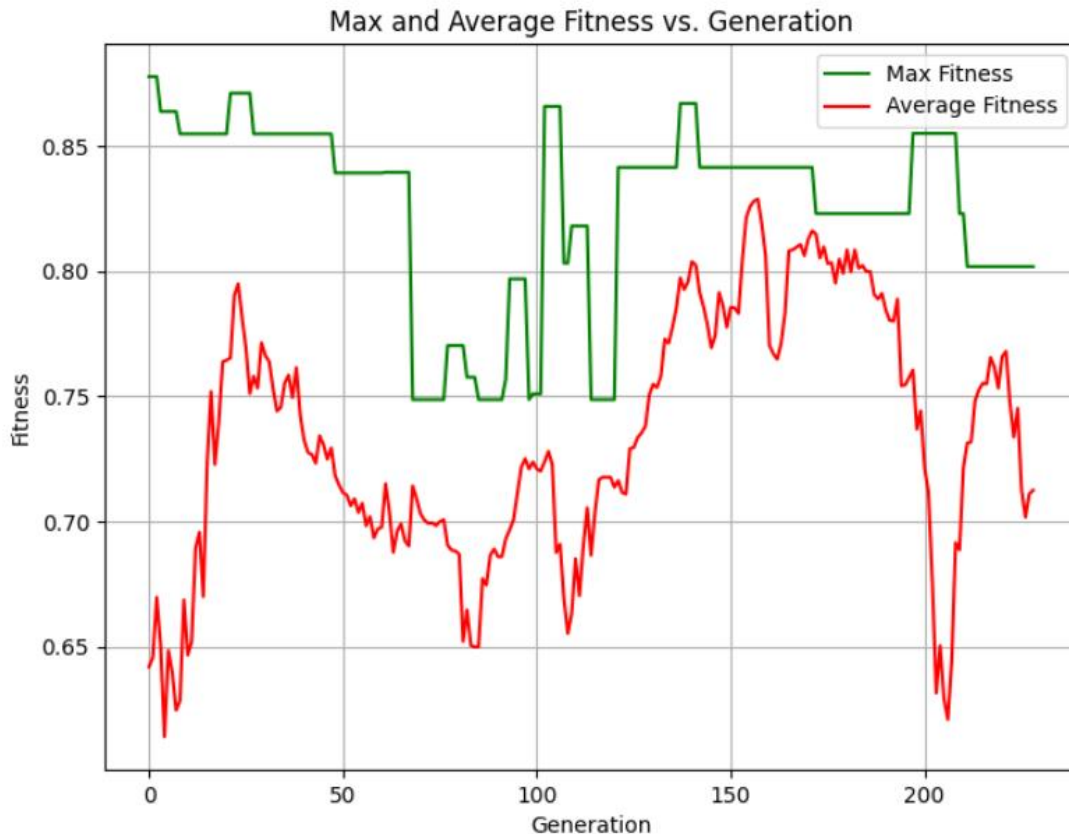
vi. RunF

E=300, I=2000, M=0.05, S=6, P=30, lr=0.001,

Also stopped because of improper convergence criteria.

```
max fitness: 0.7593789100646973 avg fitness: 0.7038863758246104
generation: 16
best chromosome of this generation: [(556, 'tanh')]
max fitness: 0.7418125867843628 avg fitness: 0.7009565571943919
Epoch 1/300
18/18 [=====] - 0s 6ms/step - loss: 1.2685 - macro_f1: 0.7505
```

```
Epoch 300/300
18/18 [=====] - 0s 6ms/step - loss: 1.2685 - macro_f1: 0.7505
Combined Train f1: 0.7506
2/2 [=====] - 0s 6ms/step - loss: 1.2650 - macro_f1: 0.7360
Test F1: 0.7360
Test Loss: 1.2650
```



This code stopped too early, so we won't be testing this chromosome.

vii. RunG-v1:

E=400, I=1000, M=0.05, S=4, P=16, lr=0.01

a) Version 1

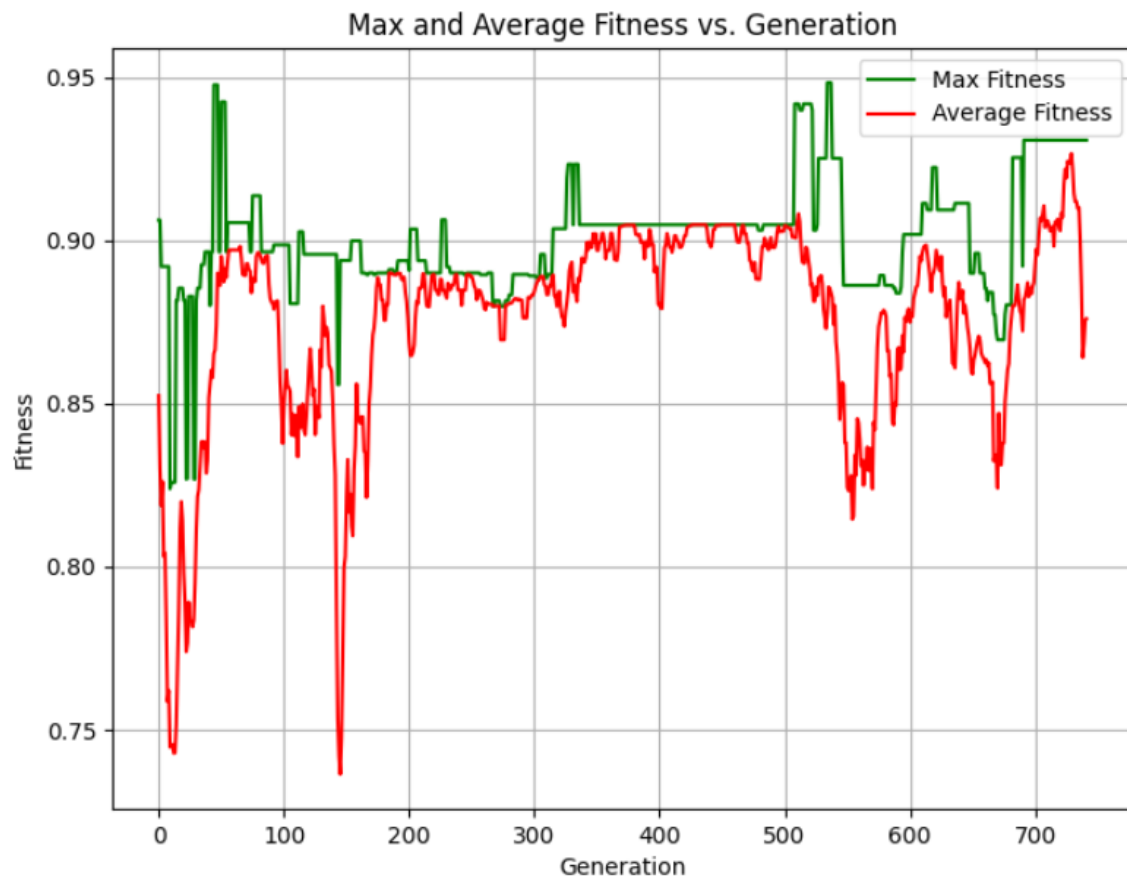
(Early stop due to unrealistic convergence condition)

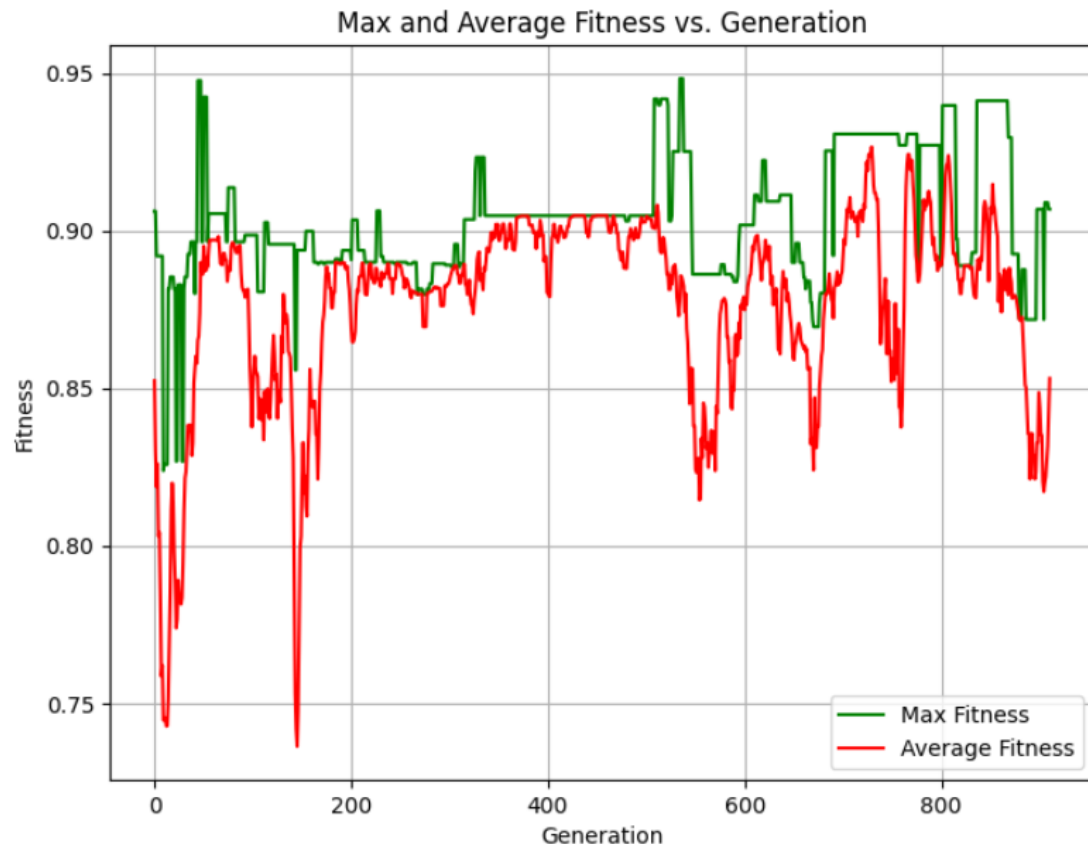
```
generation: 6
best chromosome of this generation: [(93, 'relu')]
max fitness: 0.9173239469528198 avg fitness: 0.8896438479423523
Epoch 1/400
1/103 [.....] - ETA: 33s - loss: 2.5723 - mac
```

```
103/103 [=====] - 0s 5ms/step - loss: 0.1642 - macro_f1
: 0.9548
Combined Train f1: 0.9535
1/12 [=>.....] - ETA: 0s - loss: 0.1296 - macro_f1: 0.93
12/12 [=====] - 0s 5ms/step - loss: 0.2263 - macro_f1:
0.9372
Test F1: 0.9372
Test Loss: 0.2263
>>> |
```

b) Version 2

(Still running)





We notice more stability for this run, because the mutation rate is small and S/P ratio is also small. And there is some convergence behavior, but the algorithm is still exploring.

```

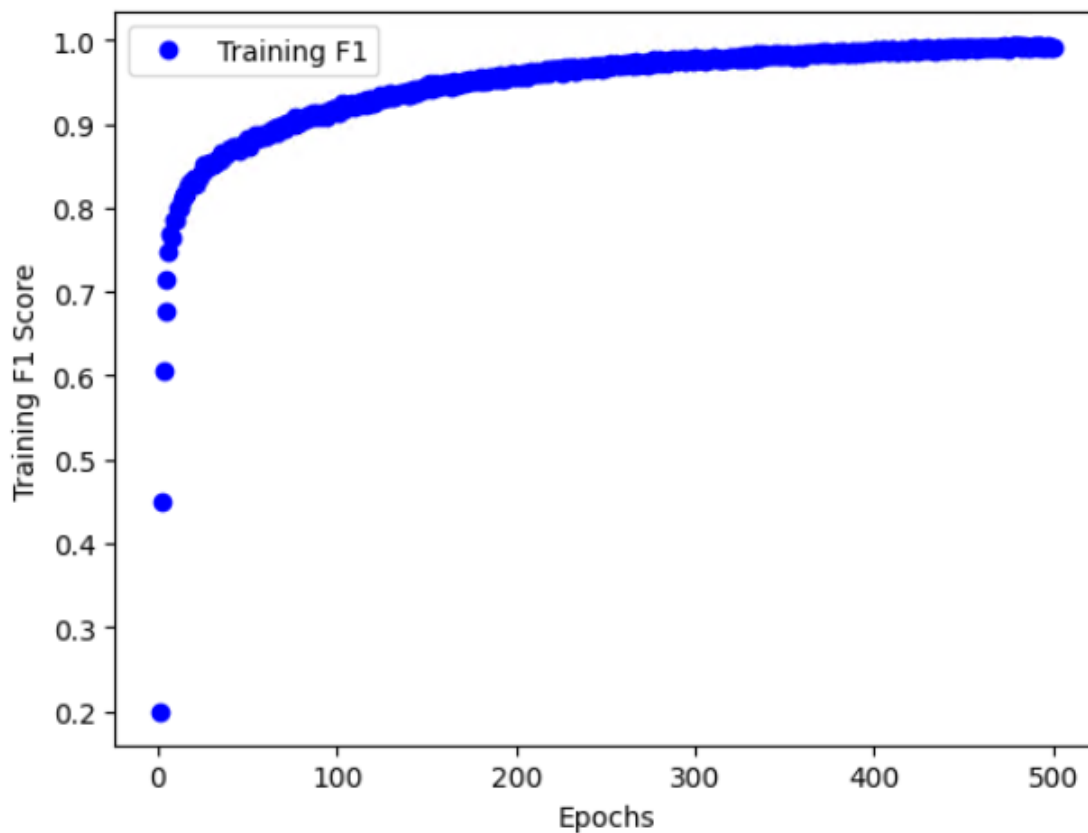
generation: 532
best chromosome of this generation: [(91, 'relu')]
max fitness: 0.9251925945281982 avg fitness: 0.8764163516461849
generation: 533
best chromosome of this generation: [(91, 'relu')]
max fitness: 0.9251925945281982 avg fitness: 0.8730200678110123
generation: 534
best chromosome of this generation: [(24, 'relu')]
max fitness: 0.9483746290206909 avg fitness: 0.8776996806263924
generation: 535
best chromosome of this generation: [(24, 'relu')]
max fitness: 0.9483746290206909 avg fitness: 0.8857974223792553
generation: 536
best chromosome of this generation: [(24, 'relu')]
max fitness: 0.9483746290206909 avg fitness: 0.8852563314139843
generation: 537
best chromosome of this generation: [(24, 'relu')]
max fitness: 0.9483746290206909 avg fitness: 0.8839804790914059
generation: 538
best chromosome of this generation: [(91, 'relu')]
max fitness: 0.9251925945281982 avg fitness: 0.8823743015527725
generation: 539
best chromosome of this generation: [(91, 'relu')]
max fitness: 0.9251925945281982 avg fitness: 0.8712626516668351

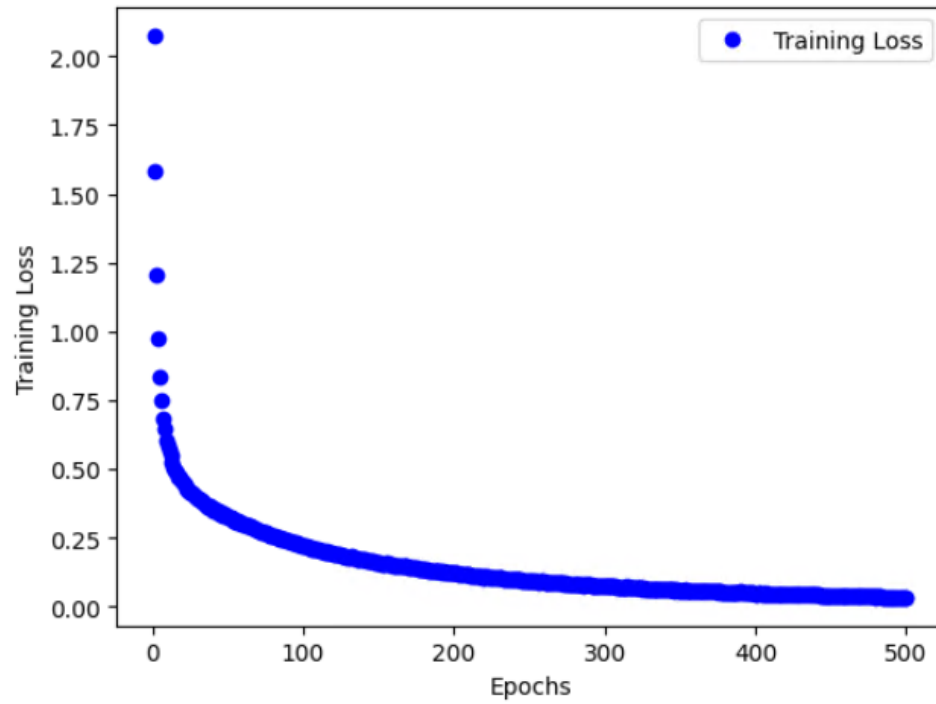
```

But the best chromosome in previous generations is (24, 'relu')

However, the last best chromosome is (29, tanh)

```
generation: 843
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.9075111709535122
generation: 844
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.9094248861074448
generation: 845
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.9080017656087875
generation: 846
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.9035281464457512
generation: 847
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.8979304879903793
generation: 848
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.8840592764317989
generation: 849
best chromosome of this generation: [(29, 'tanh')]
max fitness: 0.94129478931427 avg fitness: 0.8996536582708359
generation: 850
```





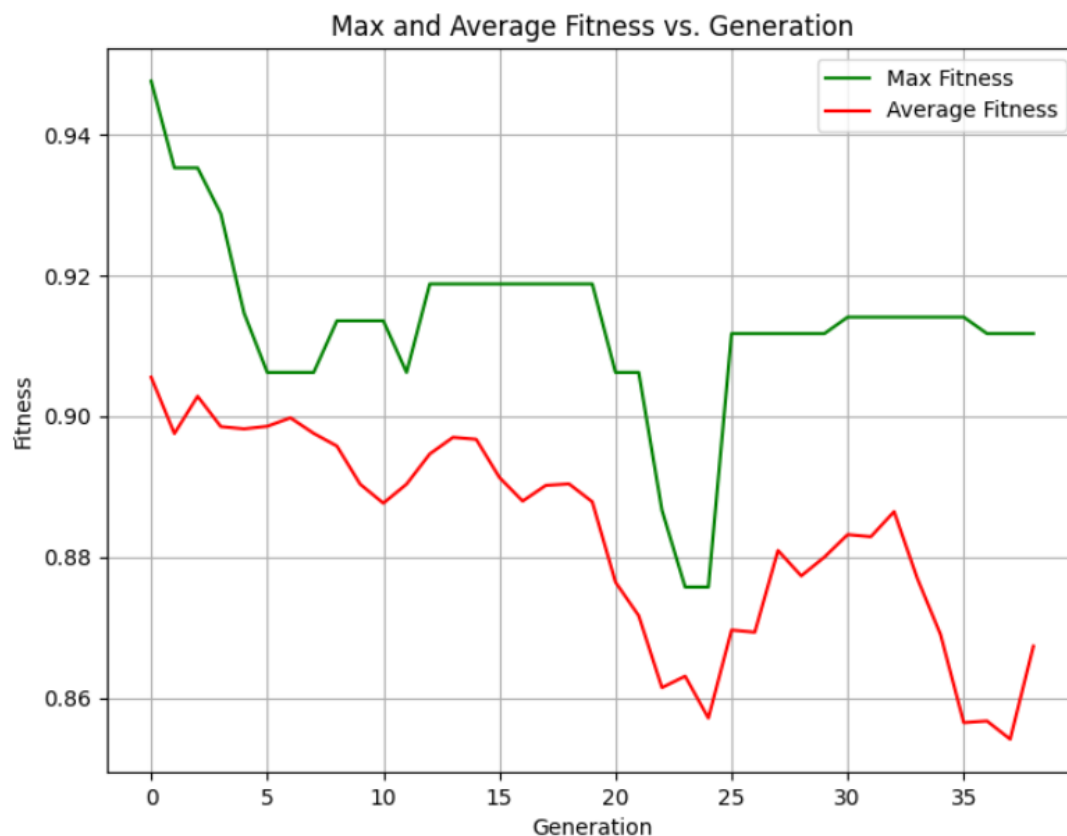
```
33/33 [=====] - 0s 1ms/step
Test F1 Score: 0.9664347447854729
33/33 [=====] - 0s 6ms/step - loss: 0.1351 - macro_f1: 0.9596
Test Loss: 0.13510747253894806
```

viii. Run H

E=600, I=200, M=0.1, S=4, P=12, lr=0.01

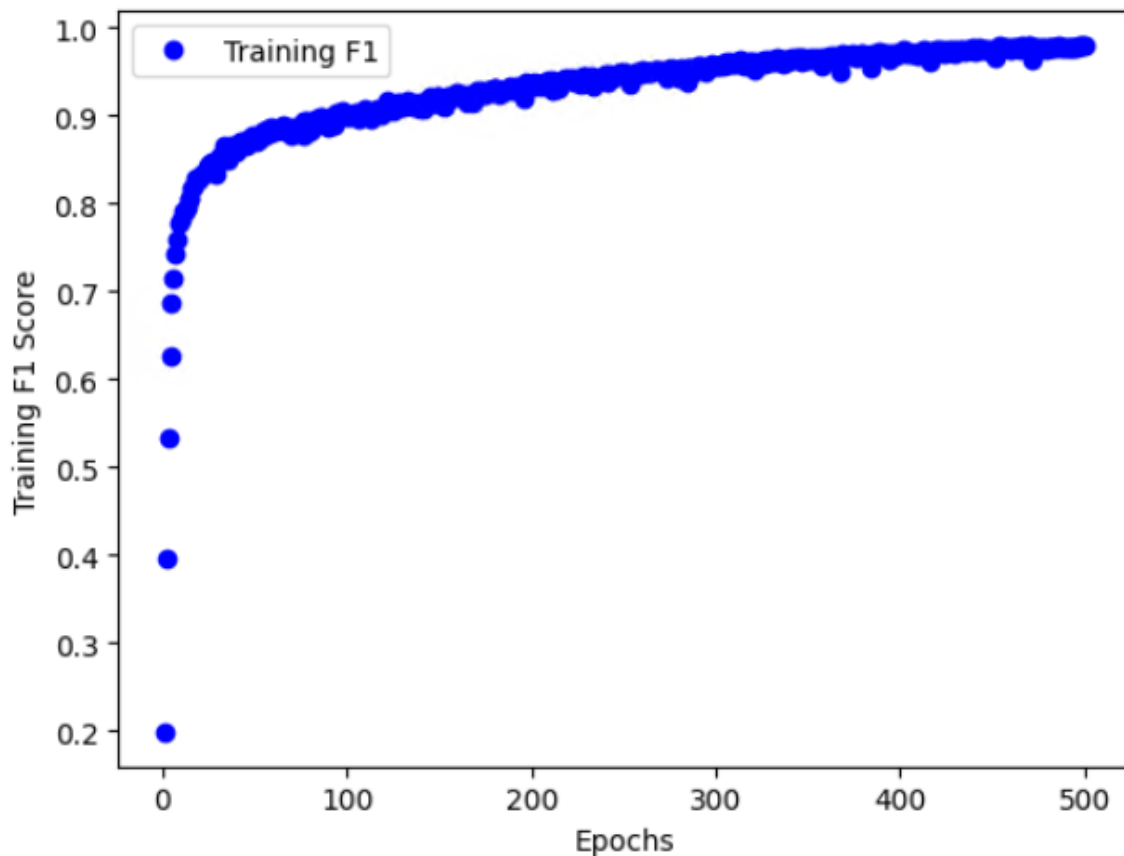
Got interrupted because of server issues or technical issues.

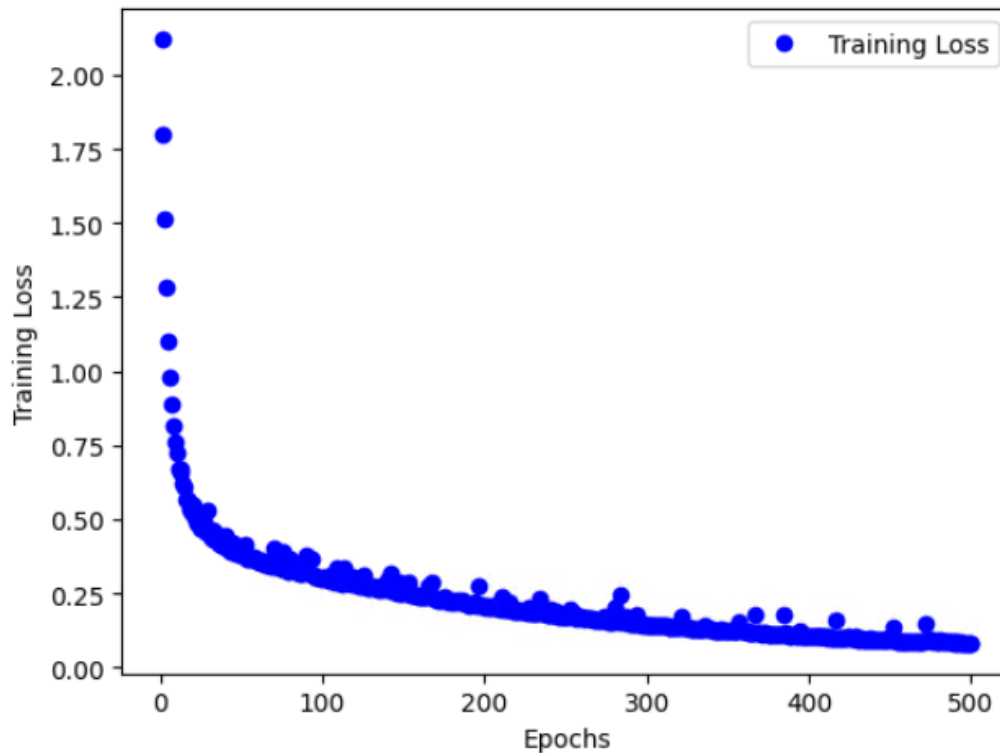
```
max fitness: 0.9188692577888228 avg fitness: 0.8687852181127207
generation: 20
best chromosome of this generation: [(247, 'relu')]
max fitness: 0.9062379598617554 avg fitness: 0.8764350910981497
generation: 21
best chromosome of this generation: [(247, 'relu')]
max fitness: 0.9062379598617554 avg fitness: 0.8716629346211752
generation: 22
best chromosome of this generation: [(536, 'relu')]
max fitness: 0.8866711258888245 avg fitness: 0.8614502002795538
generation: 23
best chromosome of this generation: [(247, 'sigmoid')]
max fitness: 0.8757379055023193 avg fitness: 0.8630780378977457
generation: 24
best chromosome of this generation: [(247, 'sigmoid')]
max fitness: 0.8757379055023193 avg fitness: 0.8571219891309738
generation: 25
client_loop: send disconnect: Connection reset
PS C:\Users\dmonz>
```



Tried to continue running after interruption, but the server crashed again.

```
generation: 40
best chromosome of this generation: [(247, 'relu')]
max fitness: 0.9062379598617554 avg fitness: 0.8785433769226074
generation: 41
best chromosome of this generation: [(55, 'tanh')]
max fitness: 0.9484955072402954 avg fitness: 0.8841585566600164
generation: 42
best chromosome of this generation: [(55, 'tanh')]
max fitness: 0.9484955072402954 avg fitness: 0.8826684753100077
generation: 43
best chromosome of this generation: [(55, 'tanh')]
max fitness: 0.9484955072402954 avg fitness: 0.8814802120129267
generation: 44
```





```
33/33 [=====] - 0s 966us/step
Test F1 Score: 0.959008938149547
33/33 [=====] - 0s 7ms/step - loss: 0.1593 - macro_f1: 0.9460
Test Loss: 0.1592969298362732
```

3. GA evaluation of hyperparameters

GA algorithm should converge to solutions (hyperparameters) that result in high validation F1 score. It is possible to achieve validation F1 score of over 95% for the given data inputs and architecture.

In summary:

runA → 23, relu

runB → 22, tanh

runC → 23, relu

runD → 134, tanh

runE → 39, relu

runG → 29, tanh

run H → 55, tanh

The output of GA depends on different variables (E, P, S, lr, M). For this reason different runs gave different outputs, however all runs gave similar outputs, so relatively small batch size between 23 and 134, and only relu or tanh.

We notice that the best combination to allow GA to converge (similar to run G), is to use small M value, and small S/P ratio, and also we must use high E to get more accurate fitness.

However using high E leads to high computational time, and causes the code to get interrupted due to limited resources, for this reason it was avoided in most runs.

If I had more time, I would test more runs with small M and S/P values until the run converges.

Out of all values hyperparameters: relu, and 23 seem best.

Possible improvements:

1. Using adaptive removal to remove a different number of old chromosomes in each generation
2. Varying mutation probability
3. Improve the optimizer (use an adaptive learning rate, and or momentum value).

Bayesian Optimization

4. BO implementation & results

Correct use of the package. Report of progress and final values that demonstrate proper searching and tuning of the hyperparameters.

- A. Report the progress output and the final values of the hyperparameters found by the algorithm.**
- B. Train the model on the training and validation data using the selected hyperparameters. Plot the training F1 score versus the training epochs.**
- C. Evaluate the model on the test data. Report the test F1 score.**

In this algorithm, I changed the activation index, so if it is 0.5, the index is `int(0.5)`, which is 0. And the activation function would be relu. For the batch size, I alternated the exponent,

and then got 2 to the power of the decimal exponent, and took the integer value of that value.

- activation = ['relu', 'sigmoid', 'tanh'][int(activation_ind)]
- batch_size = int(2 ** batch_size_exp)
- pbounds={'activation_ind': (0, 2.999), 'batch_size_exp': (4, 10)},

The default learning rate for the SGD optimizer in Keras is typically set to 0.01. However, in some codes, I changed the learning rate to 0.001.

Note that initially, I set the number of iterations for the runs, hoping to explore that many iterations, however, I had so many interruptions with this package. (Also due to the spawning issue of the servers). So I couldn't run all the iterations for most of the runs.

ix. Run A

E=100, lr=0.01

(Was interrupted due to server crash)

iteration	target	activation index	batch_size exponent

1	0.8472	0.4623	8.44
2	0.8751	0.7897	7.202
3	0.8044	0.04371	9.512
4	0.9296	2.701	4.201
5	0.9228	2.87	4.823
6	0.9298	0.0	4.0
7	0.8923	1.332	4.0
8	0.9034	0.0	5.33
9	0.8691	2.999	7.645
10	0.776	2.999	10.0
11	0.9374	2.999	4.0
12	0.8937	2.999	6.066
13	0.9255	2.999	4.283
14	0.8718	1.722	5.538
15	0.9145	0.0	4.527
16	0.876	0.009181	6.567
17	0.9321	2.346	4.02
18	0.9364	2.706	4.0
19	0.9346	2.835	4.0
20	0.8185	1.984	7.0
21	0.9334	0.4783	4.0
22	0.9137	0.7499	4.832
23	0.9023	2.983	5.335
24	0.9152	2.215	4.735
25	0.9272	0.4515	4.316
26	0.8445	0.657	5.686
27	0.8263	0.0	7.534
28	0.6202	1.724	8.572
29	0.8722	2.999	6.869

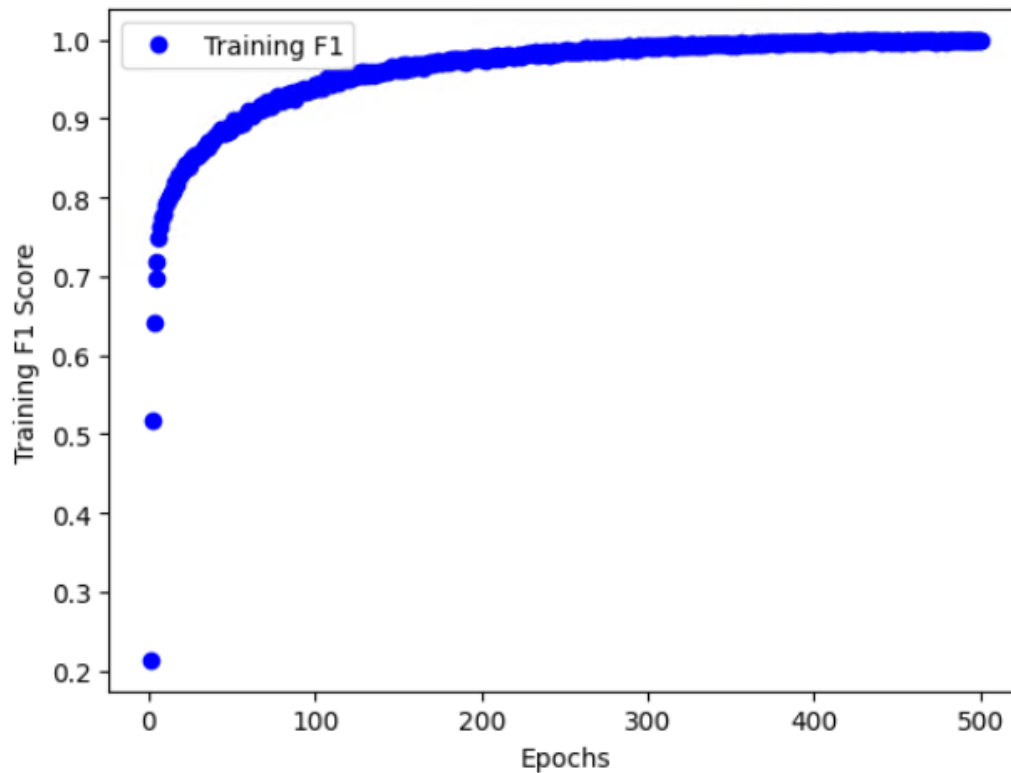
30	0.885	1.412	4.709	
31	0.901	2.44	5.337	
32	0.9323	0.2651	4.0	
33	0.9168	0.8065	4.299	
34	0.9159	0.3287	4.824	
35	0.9113	2.568	4.597	
36	0.9314	2.126	4.286	

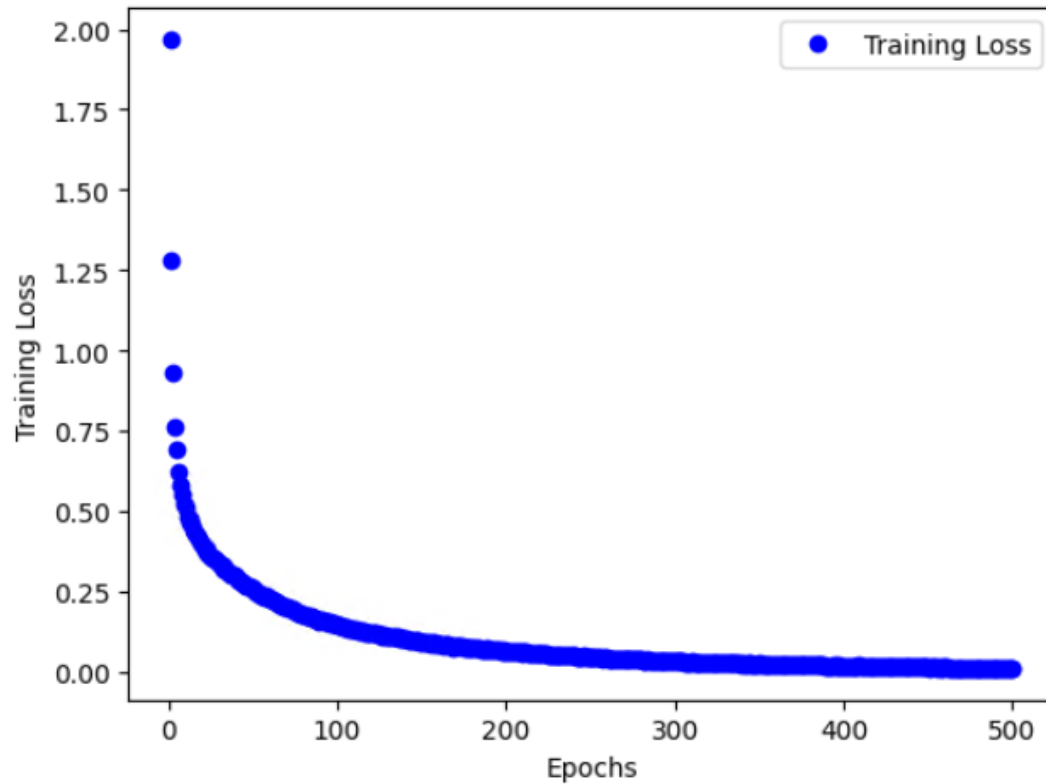
The best hyperparameter run of this set is:

| 11 | 0.9374 | 2.999 | 4.0 |

Which refers to {"tanh", batch size = 16}

Trained model with E= 500, lr=0.01:





```
33/33 [=====] - 0s 1ms/step
Test F1 Score: 0.970864658926496
33/33 [=====] - 0s 6ms/step - loss: 0.1349 - macro_f1: 0.9671
Test Loss: 0.13493657112121582
```

x. Run B

E=300, lr=0.01

(I ran this locally so it was super slow, I ran it for almost 20 hours and this is how many iterations I got)

iter	target	activa...	batch_...
1	0.8883	0.4623	8.44
2	0.896	0.7897	7.202
3	0.87	0.04371	9.512
4	0.9592	2.701	4.201
5	0.9522	2.87	4.823
6	0.9524	0.0	4.0
7	0.9265	1.434	4.0
8	0.8566	2.999	10.0
9	0.9425	0.0	5.241

10	0.9609	2.999	4.0	
11	0.9099	2.999	6.63	
12	0.9571	2.999	4.342	
13	0.9026	1.869	5.292	
14	0.9539	0.0	4.545	
15	0.9589	2.548	4.0	
16	0.9582	2.803	4.0	
17	0.8811	2.999	8.031	
18	0.9086	0.0	6.362	
19	0.9585	2.276	4.379	
20	0.9579	2.546	4.507	
21	0.9565	2.204	4.0	
22	0.9557	2.421	4.222	
23	0.9467	0.6151	4.721	
24	0.934	2.999	5.583	
25	0.9587	0.499	4.0	
26	0.6716	1.399	10.0	
27	0.8911	0.0	7.669	
28	0.8709	1.964	7.114	
29	0.9197	0.7065	5.743	
30	0.9475	0.3362	4.302	

The best hyperparameters for this set are:

| 10 | 0.9609 | 2.999 | 4.0 |
Which refer to activation = "tanh", batchsize = 16.

This was tested in run A. Same results on the training model.

xi. Run C

E=50, lr=0.001

(Accidentally got interrupted, it took about 20 hours to run these iterations, lost of print out of optimization loop failure which is common due to resource limitation sometimes, so I couldn't track all iterations)

288	0.8939	2.879	4.477	
289	0.8969	0.3538	4.002	
290	0.9073	2.326	4.033	
291	0.8864	0.2614	4.345	
292	0.9033	0.2993	4.038	
293	0.9072	2.621	4.004	
294	0.9085	0.2759	4.024	
295	0.9132	2.341	4.049	

296	0.9045	2.872	4.01	
297	0.9037	2.71	4.074	
298	0.8222	2.917	8.269	
299	0.907	2.403	4.029	
300	0.9067	2.599	4.071	
301	0.9134	2.731	4.056	
302	0.9064	2.363	4.053	
303	0.7396	0.3537	9.377	
304	0.9122	2.818	4.081	
305	0.9132	2.767	4.029	
306	0.8381	0.2558	7.351	
307	0.9006	2.739	4.205	
308	0.888	0.1869	4.346	
309	0.8834	0.1952	4.326	
310	0.8807	0.213	4.347	
311	0.9052	2.35	4.076	
312	0.908	2.375	4.027	
313	0.8921	0.329	4.042	
314	0.7649	1.887	6.597	
315	0.4839	1.227	9.041	
316	0.9027	0.2592	4.003	
317	0.8862	0.08283	4.515	
318	0.9044	2.861	4.083	
319	0.555	1.327	8.074	
320	0.8902	2.609	4.095	
321	0.9088	2.778	4.007	
322	0.9071	2.781	4.059	
323	0.8917	2.371	4.102	
324	0.897	2.844	4.102	
325	0.2971	1.733	9.191	
326	0.8702	1.43	4.374	
327	0.9149	2.752	4.079	
328	0.888	2.752	4.113	
329	0.6694	1.269	7.527	
330	0.9117	2.867	4.054	
331	0.9074	2.391	4.004	
332	0.9042	2.573	4.087	
333	0.8992	2.711	4.1	
334	0.2108	1.408	9.898	
335	0.8975	2.778	4.134	
336	0.9005	2.909	4.46	

337	0.9043	0.2559	4.051	
338	0.8824	2.779	4.09	
339	0.9063	0.776	4.047	
340	0.9102	2.833	4.008	
341	0.8989	0.2127	4.49	

```

as cancelled
2023-12-08 21:04:15.574379: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:04:15.570534: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:50.446066: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:51.239943: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:52.672992: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:52.940611: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:53.045493: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:53.117009: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:53.202781: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:53.519542: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:05:53.571699: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
| 289 | 0.8939 | 2.679 | 4.477 |
2023-12-08 21:10:03.838859: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.139539: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.195802: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.298536: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.526017: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.729311: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:04.770050: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:25.158033: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:25.576173: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:25.708644: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:25.931229: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:26.164429: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:10:26.678209: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
| 289 | 0.8969 | 0.3538 | 4.002 |
2023-12-08 21:12:33.228685: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:12:33.335612: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:12:34.118729: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:12:34.299003: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:12:34.740255: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:19.309197: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:19.429807: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:19.609926: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:19.672879: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:19.865967: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:21.347927: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:21.561384: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:21.675685: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:21.956321: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.001524: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.138451: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.182356: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.235891: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.291850: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.635851: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:22.857358: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:14:23.431630: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:53.307525: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:53.359799: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:53.619870: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:53.798473: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:53.855789: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:59.017263: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled
2023-12-08 21:15:59.121641: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCELLED: Operation was cancelled

```

(This error occurs with many users, see: <https://discuss.tensorflow.org/t/optimization-loop-failed-cancelled-operation-was-cancelled/1524/45>)

None of these hyperparameters is purple, which means that a previous iteration had better performance. However, I cannot track it down from the terminal, due to the clutter of printouts by tensorflow. However, out of these outputs. The best hyperparameter set is:

301 | 0.9134 | 2.731 | 4.056 |

Which refers to activation = "tanh", batch_size = 16.

Same result as in runs A, and B. Got tested in both.

xii. Run D

E=20, lr=0.01

```

started optimizing
| iter      | target    | activa... | batch... |
|-----|-----|-----|-----|
| 1          | 0.6931    | 0.4623    | 8.44     |
| 2          | 0.7959    | 0.7897    | 7.202    |
| 3          | 0.537     | 0.04371   | 9.512    |
| 4          | 0.8694    | 2.701     | 4.201    |
| 5          | 0.8618    | 2.87      | 4.823    |
| 6          | 0.8532    | 0.0       | 4.0      |
| 7          | 0.7641    | 2.999     | 7.748    |
2023-12-08 10:02:50.533218: W tensorflow/core/data/r
as cancelled
2023-12-08 10:03:00.961047: W tensorflow/core/data/r
as cancelled
| 8          | 0.7923    | 1.303     | 4.963    |
| 9          | 0.8134    | 0.0       | 5.743    |
| 10         | 0.4226    | 2.999     | 10.0     |
2023-12-08 10:04:22.667591: W tensorflow/core/data/r
as cancelled
| 11         | 0.7699    | 2.999     | 6.263    |
| 12         | 0.8698    | 2.999     | 4.0      |
2023-12-08 10:07:59.094423: W tensorflow/core/data/r
as cancelled
| 13         | 0.8344    | 1.288     | 4.0      |
| 14         | 0.6217    | 0.0       | 7.038    |
| 15         | 0.432     | 1.84      | 7.151    |

```

See appendix for the full table (about 690 iterations, took also about 21 hours)

The best iteration was at iteration 67:

```
| 67      | 0.8775    | 2.816     | 4.067    |
```

Which corresponds to hyperparameters = activation = “tanh”, batch_size =16.

Same result as runs A, B, C.

xiii. Run E

E=200, lr=0.001

c) Version 1 (got interrupted)

iter	target	activa...	batch_...
2023-12-08 08:07:42.795252: I tensorflow/stream_executor... will only be logged once.			
1	0.8787	0.4623	8.44
2	0.8857	0.7897	7.202
3	0.8511	0.04371	9.512
4	0.9397	2.701	4.201
5	0.9443	2.87	4.823
6	0.9489	0.0	4.0
7	0.9027	1.274	4.638
8	0.8983	2.999	6.6
9	0.8293	2.999	10.0
10	0.9203	0.0	5.158
11	0.9491	2.999	4.0

d) Version 2 (also got interrupted)

iter	target	activa...	batch_...
2023-12-08 09:49:33.496861: I tensorflow/stream_executor... will only be logged once.			
1	0.8746	0.4623	8.44
2	0.8922	0.7897	7.202
3	0.8479	0.04371	9.512
4	0.9448	2.701	4.201
5	0.9442	2.87	4.823
6	0.9548	0.0	4.0
7	0.9092	1.178	4.0
8	0.9221	0.0	5.202
9	0.8914	2.999	6.839
10	0.8373	2.999	10.0
11	0.9508	2.999	4.0

So, the best two sets of hyperparameters:

Set1: activation = "tanh", batch_size = 16

Set2: activation = "relu", batch_size = 16

Results on training the model, with relu and batch size = 16, and E=500, lr=0.01:

xiv. Run F

E=300, lr=0.001

iter	target	activa...	batch_...
2023-12-08 21:34:32.405287: I tensorflow/stream_executor/matrix multiplication. This will only be logged once.			
1	0.8499	1.818	7.945
2	0.8588	0.9214	9.772
3	0.8437	1.607	7.958
4	0.8878	2.997	8.506
5	0.8259	1.615	8.473
6	0.8845	2.999	7.932
7	0.8681	2.999	9.634
8	0.9084	2.999	6.67
9	0.9439	2.999	5.693
10	0.9515	2.211	4.818
11	0.956	2.989	4.049
2023-12-08 23:01:14.856045: W tensorflow/core/data/ as cancelled			
12	0.9503	0.05664	4.0
13	0.9258	0.0	5.464
14	0.9265	1.338	4.0
2023-12-09 00:03:24.443068: W tensorflow/core/data/ as cancelled			
2023-12-09 00:03:24.670060: W tensorflow/core/data/ as cancelled			
2023-12-09 00:03:24.743032: W tensorflow/core/data/ as cancelled			
15	0.947	2.999	4.766
16	0.8927	1.633	5.709

Also got interrupted.

```
2023-12-09 00:25:22.250539: W tensorflow/core/data/ as cancelled
client_loop: send disconnect: Connection reset
```

Best set of hyperparameters:

Activation function = “tanh”, batch_size = 16

Same result as in runs previously.

xv. Run G

E=400, lr=0.01

The terminal got cluttered. These are the highest iterations I could track.

```
2023-12-08 23:02:08.602335: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:02:09.278232: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:02:09.463589: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:02:10.067798: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
| 11 | 0.9608 | 2.999 | 4.287 |
2023-12-08 23:03:37.844113: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:03:38.032988: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:03:38.368648: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA
2023-12-08 23:03:39.718377: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CA

2023-12-09 01:07:02.605339: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:07:03.155555: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:07:03.231121: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
| 15 | 0.9615 | 0.8208 | 4.0 |
2023-12-09 01:07:39.878381: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:07:39.945520: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:08:31.431561: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:08:32.206342: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:08:33.015365: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
2023-12-09 01:08:33.041057: W tensorflow/core/data/root_dataset.cc:266] Optimization loop failed: CANCEL
```

Best set of hyperparameters:

Activation function = “relu”, batch_size = 16

Same as run E.

xvi. **Run H**

E=600, lr=0.01

```

| iter | target | activa... | batch... |
-----
2023-12-08 23:25:22.139055: I tensorflow/stream_executor
TensorFloat-32 will be used for the matrix multiplication once.
| 1 | 0.9051 | 1.26 | 6.179 |
| 2 | 0.9363 | 0.5544 | 7.11 |
| 3 | 0.8839 | 0.02581 | 9.814 |
2023-12-08 23:49:36.318185: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
| 4 | 0.899 | 2.403 | 8.544 |
2023-12-09 00:14:39.663507: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 00:19:23.834511: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 00:24:42.871445: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 00:25:06.583524: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 00:25:18.716606: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 00:25:18.774004: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
| 5 | 0.9669 | 2.014 | 4.178 |
2023-12-09 01:00:11.590221: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
2023-12-09 01:05:21.355441: W tensorflow/core/data/re
tion loop failed: CANCELLED: Operation was cancelled
| 6 | 0.9592 | 2.053 | 4.196 |

```

Best set of hyperparameters:

Activation function = “tanh”, batch_size = 18

5. BO evaluation of hyperparameters

BO algorithm should converge to solutions (hyperparameters) that result in high validation F1 score. It is possible to achieve validation F1 score of over 95% for the given data inputs and architecture.

The highest fitness value I obtained out of all runs was:

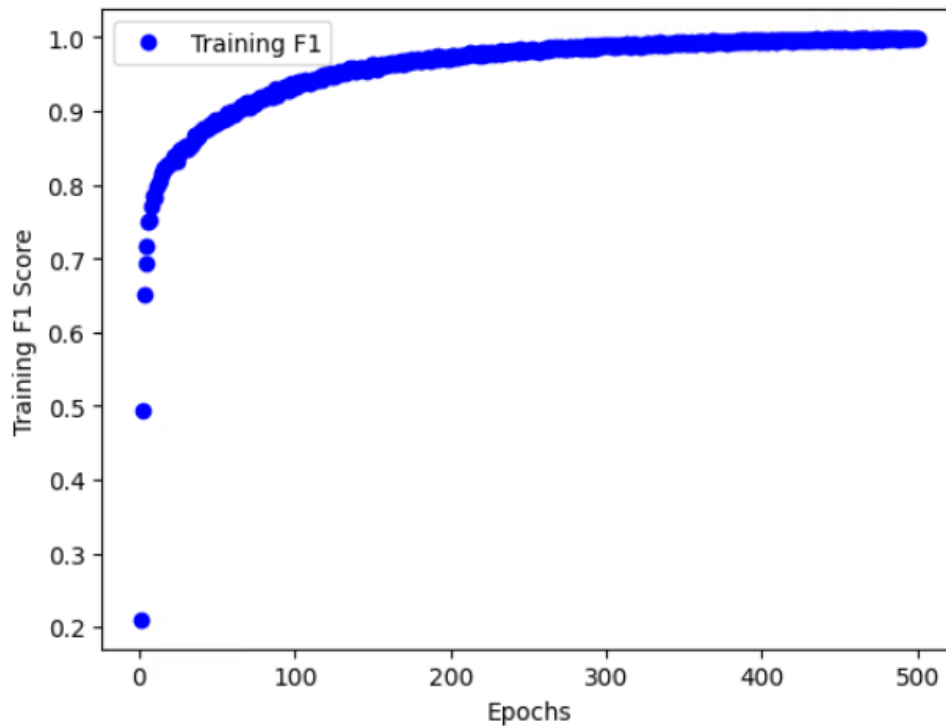
0.9669 using activation function = “tanh”, and batch_size =18. However, almost all runs resulted in almost the same set: either “relu” or “tanh” as an activation function, and either batch size=16 or 18. So even with different E values and learning rate values, and with extensive generations exploration (as in run C), these sets are the best.

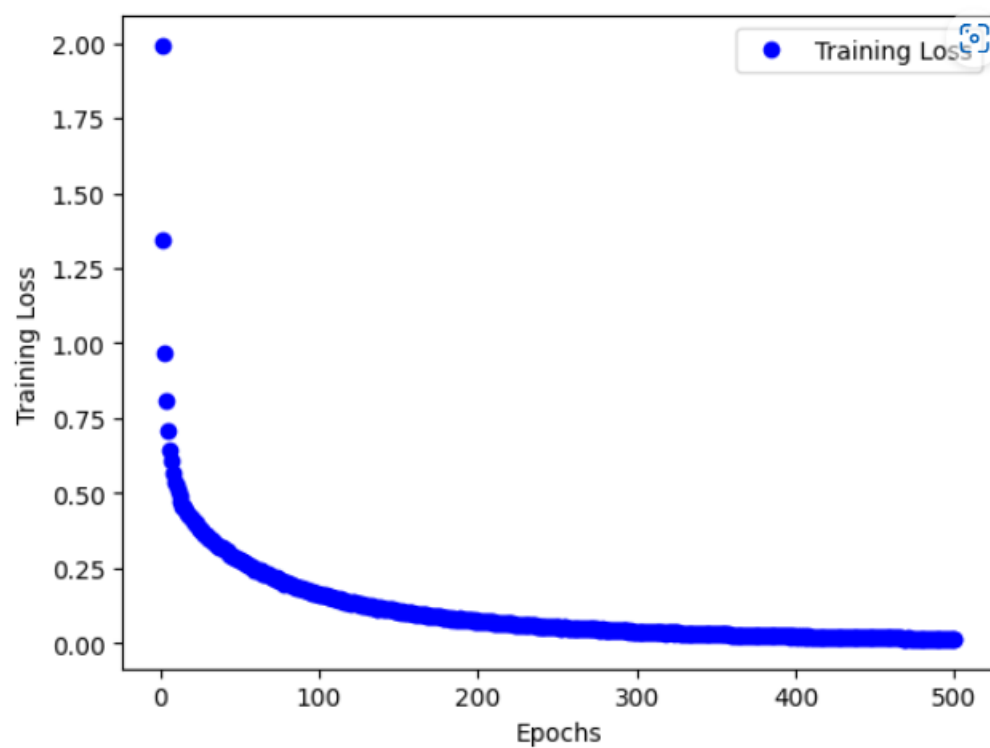
For this reason, I will test these two sets on the data with $E=500$ and $lr = 0.01$.

We notice that although E and lr fluctuated, all runs had similar results. This shows the robustness of BO.

(Note that due to interruptions, I wasn't able to obtain `optimizer.res` from BO, so I can perhaps plot the fitness versus iterations, although it is not required.)

a) Activation function = "tanh", batch_size = 18





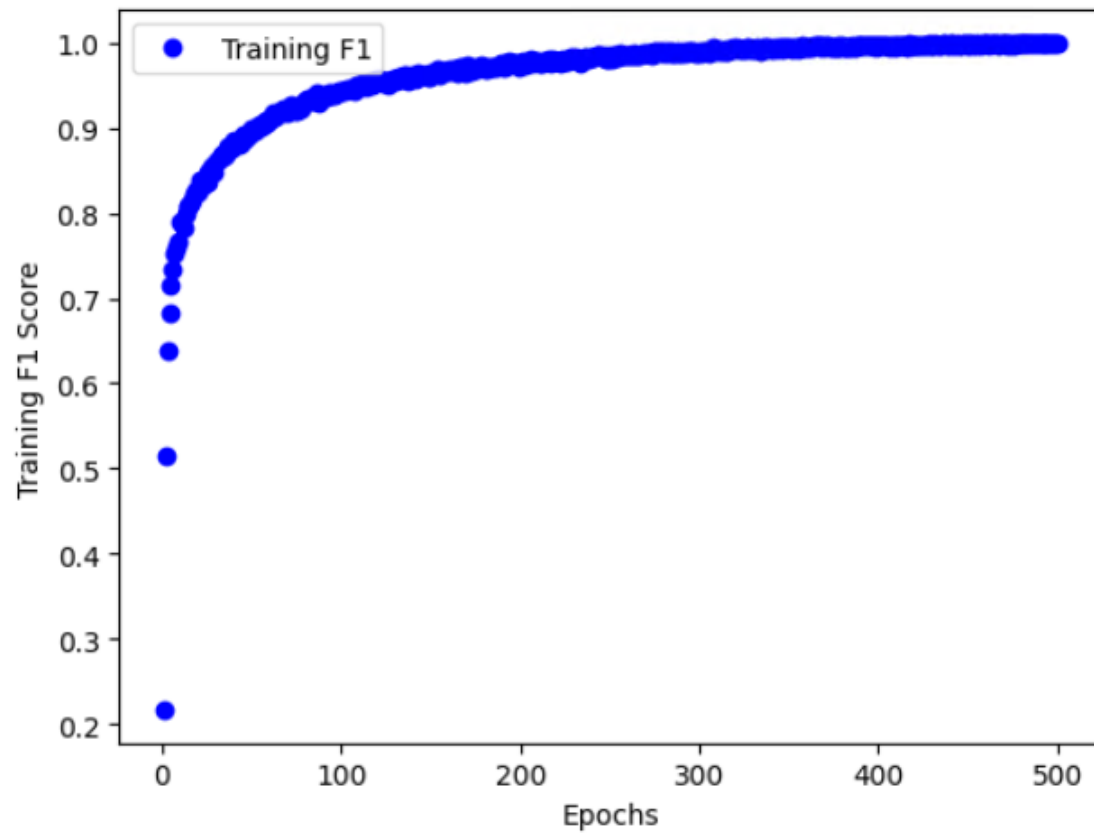
33/33 [=====] - 0s 1ms/step

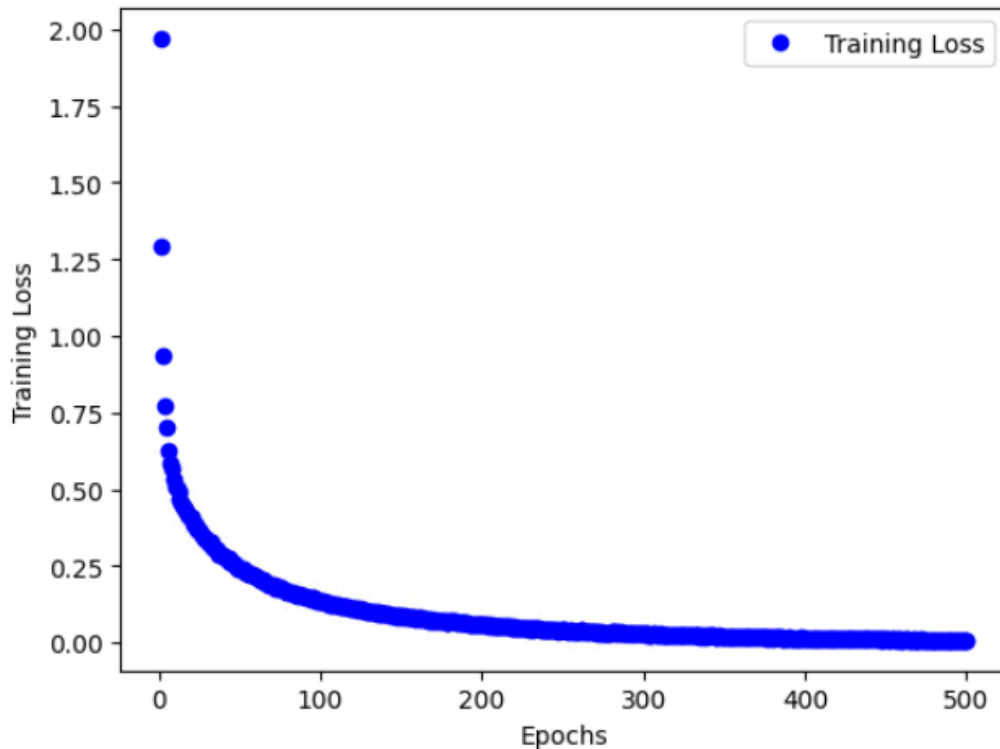
Test F1 Score: 0.9701024331227538

33/33 [=====] - 0s 5ms/step - loss: 0.1353 - macro_f1: 0.9649

Test Loss: 0.1352679580450058

b) Activation function = "Relu", batch_size =16





33/33 [=====] - 0s 1ms/step

Test F1 Score: 0.9691037211008837

33/33 [=====] - 0s 6ms/step - loss: 0.1396 - macro_f1: 0.9664

Test Loss: 0.13961951434612274

6. Discuss BO and GA

E. Compare the hyperparameters chosen by the Genetic Algorithm and the Bayesian Optimization method.

I ran 8 different runs at least for each algorithm. In order to be able to compare the results, I tried to match the hyperparameters in runs. The results between BO and GA didn't match exactly for any run, however this could be due to technical issues that didn't let GA to converge properly and the several interruptions of BO. However, both algorithms had close enough values. All runs in BO gave the best batch size as either 16 or 18. While several runs in GA gave the best batch size as 22, 23, 29. So there is some matching.

The best chromosome out of GA runs is (23, relu), and the best chromosome out of BO runs is (tanh, 18), they both give 0.97, however the former is only slightly higher in terms of macro-f1 test result. So both algorithms give reliable and good hyperparameters. However, based on the limited runs I conducted, GA gives slightly better results.

F. Briefly discuss the pros and cons of the two methods.

Genetic Algorithm Method for Hyper tuning hyperparameters:

Pros:

- ✚ It is good at exploring a wide search space by maintaining a population of solutions and evolving them over generations. This helps in finding global optima.
- ✚ It can possibly handle parallel evaluations of different solutions, allowing multiple hyperparameter configurations to be tested simultaneously. However, I didn't utilize this feature in this hw.
- ✚ It is versatile and adaptable to various problem types, making it suitable for non-differentiable or complex search spaces.
- ✚ It is suitable for discrete hyperparameters or continuous hyperparameters.
- ✚ It is a well-trusted method that outperforms several other methods and it is adopted in different fields.

Cons:

- It can take longer to converge to optimal solutions compared to more focused methods. It might need more iterations to converge. And it highly depends on its hyperparameters.
- It might require more computational resources as it explores a larger search space with multiple evaluations.
- Different hyperparameter sets can lead to very different solutions.
- In terms of implementation, it needs a lot of hyperparameter tuning itself in order to get good results.
- Convergence becomes difficult with a big population or mutation rate.
- Since the final result of the best chromosome depends on the final population; one might find the final chromosome to be not as good as other chromosomes in previous generations, as we have seen in some of my runs. However, this issue could be solved with proper convergence and higher computational time. A better improvement of GA is tabu-search.

Bayesian Optimization Method for Hyper tuning hyperparameters:

Pros:

- ✚ It focuses on the most promising regions of the search space based on past evaluations, which can lead to faster convergence.
- ✚ It constructs a probabilistic model of the objective function, allowing for a more informed search and exploitation of the space.
- ✚ It often requires fewer evaluations to find a good solution, making it more sample efficient.
- ✚ It is more robust to variations in number of epochs and learning rate of the model. It can still find the best solution.

Cons:

- It might struggle when dealing with categorical or discrete hyperparameters, although adaptations exist.
- It can be sensitive to noisy evaluations and might struggle to distinguish between actual improvements and random fluctuations.
- It can be very computationally intensive.
- The implementation is a black box so it is more difficult to customize.

Comparison:

For complex search spaces or non-differentiable problems, GAs might be more suitable due to their global search capabilities. On the other hand, BO might be preferred when computational resources are limited, as it's often more sample efficient. If the hyperparameters include categorical or discrete values, GAs might be more appropriate, whereas BO suits continuous spaces better.

Notes:

**** Please make sure you use the correct packages when running my codes. For example, I tried to use google colab and Kaggle, but due to versions mismatch and even the python versions are different, I got errors in data splitting and SoftMax respectively. Although when I run the code in my local environment or the server (deepdish) the codes run fine with no errors.**

So, I don't recommend you use google colab or Kaggle to run my code.

Acknowledgements:

- Chatgpt
- TA
- Lectures

Appendix

xvii. P2- BO - Run D

Progress output table.

iter	target	activa...	batch_...	

1	0.6931	0.4623	8.44	
2	0.7959	0.7897	7.202	
3	0.537	0.04371	9.512	

4	0.8694	2.701	4.201	
5	0.8618	2.87	4.823	
6	0.8532	0.0	4.0	
7	0.7641	2.999	7.748	
8	0.7923	1.303	4.963	
9	0.8134	0.0	5.743	
10	0.4226	2.999	10.0	
11	0.7699	2.999	6.263	
12	0.8698	2.999	4.0	
13	0.8344	1.288	4.0	
14	0.6217	0.0	7.038	
15	0.432	1.84	7.151	
16	0.8576	0.2714	4.784	
17	0.8028	2.254	5.35	
18	0.7423	0.8689	5.875	
19	0.844	0.6207	4.0	
20	0.8758	2.002	4.004	
21	0.7935	2.993	5.491	
22	0.7096	2.074	4.602	
23	0.1904	1.714	8.86	
24	0.8236	0.003061	5.213	
25	0.7997	0.6624	5.158	
26	0.7995	2.999	7.052	
27	0.8449	0.0	4.465	
28	0.8526	0.7919	4.505	
29	0.8433	2.995	4.364	
30	0.8622	2.441	4.005	
31	0.734	0.0	7.998	
32	0.6928	1.804	5.843	
33	0.739	0.6296	7.718	
34	0.8288	0.3654	4.331	
35	0.809	0.6447	6.567	
36	0.8385	1.709	4.0	
37	0.8451	2.673	5.116	
38	0.8629	2.8	4.012	
39	0.7017	2.682	4.604	
40	0.8589	0.7084	4.537	
41	0.8528	2.982	5.064	
42	0.6655	2.999	8.538	
43	0.867	0.7066	4.545	
44	0.8459	0.264	4.782	
45	0.8536	0.5348	4.801	

46	0.8468	0.9203	4.767	
47	0.1209	1.782	9.919	
48	0.8174	1.3	4.445	
49	0.5945	1.392	6.777	
50	0.7846	0.3358	5.459	
51	0.8229	0.2422	6.216	
52	0.8412	2.549	5.81	
53	0.7621	0.7917	7.205	
54	0.7539	0.74	7.163	
55	0.793	0.7918	7.153	
56	0.8652	0.7795	4.552	
57	0.6917	0.7378	4.608	
58	0.4699	2.945	9.085	
59	0.8372	1.986	4.054	
60	0.8703	2.046	4.033	
61	0.763	0.1331	7.636	
62	0.8534	2.749	4.233	
63	0.8505	0.2616	4.778	
64	0.8431	0.328	4.74	
65	0.8342	0.2677	4.709	
66	0.8472	2.772	4.158	
67	0.8775	2.816	4.067	
68	0.8577	2.027	4.908	
69	0.44	1.457	7.705	
70	0.8612	2.874	4.041	
71	0.859	2.953	4.053	
72	0.8622	2.737	4.064	
73	0.8477	2.873	4.122	
74	0.8497	2.671	4.28	
75	0.8471	0.3582	4.805	
76	0.5797	0.8526	9.033	
77	0.7374	2.432	8.094	
78	0.8463	2.704	4.336	
79	0.8379	2.698	4.123	
80	0.8422	0.4493	4.784	
81	0.8503	0.4839	4.851	
82	0.8574	2.617	4.194	
83	0.854	2.976	4.523	
84	0.8135	2.972	4.449	
85	0.8168	2.91	4.894	
86	0.8422	2.944	4.796	
87	0.8583	2.88	4.745	

88	0.3259	1.566	7.832	
89	0.8631	2.524	4.057	
90	0.845	2.438	4.092	
91	0.8427	2.528	4.149	
92	0.851	2.605	4.087	
93	0.7108	2.949	4.641	
94	0.8646	2.641	4.009	
95	0.8522	2.787	4.788	
96	0.8603	0.8726	4.552	
97	0.7332	1.097	5.609	
98	0.8246	0.9064	4.66	
99	0.8699	2.344	4.028	
100	0.8427	2.332	4.107	
101	0.4769	0.6656	9.055	
102	0.8266	1.382	4.263	
103	0.8611	0.6183	4.571	
104	0.868	2.26	4.009	
105	0.4857	2.464	7.507	
106	0.8657	2.175	4.049	
107	0.806	2.667	7.141	
108	0.6404	0.3408	8.742	
109	0.8297	2.211	4.128	
110	0.8504	0.6215	4.002	
111	0.8277	0.6595	4.094	
112	0.8519	0.7009	4.021	
113	0.8516	0.5555	4.071	
114	0.805	0.5388	4.665	
115	0.8362	2.095	4.113	
116	0.8658	2.701	4.199	
117	0.8421	2.009	5.008	
118	0.7978	1.748	4.762	
119	0.8431	2.117	4.962	
120	0.7939	1.929	4.923	
121	0.8628	2.12	4.848	
122	0.322	1.11	7.881	
123	0.7189	2.308	8.378	
124	0.8604	0.9274	4.823	
125	0.8539	2.033	4.804	
126	0.8445	0.9295	5.004	
127	0.5905	1.819	6.569	
128	0.8504	0.8521	4.913	
129	0.8231	2.602	5.388	

130	0.7967	1.035	4.8	
131	0.2116	1.393	8.8	
132	0.8479	0.9865	4.917	
133	0.8383	2.866	5.112	
134	0.8252	2.967	5.175	
135	0.8449	2.769	4.994	
136	0.8065	2.711	4.898	
137	0.8311	2.895	4.276	
138	0.8486	2.886	5.012	
139	0.4474	1.196	7.652	
140	0.8382	0.7972	5.029	
141	0.5044	2.237	9.362	
142	0.8499	2.995	4.166	
143	0.806	2.587	5.323	
144	0.8361	2.748	5.211	
145	0.812	2.825	4.394	
146	0.6416	1.419	6.139	
147	0.7566	0.8803	7.454	
148	0.8531	0.6903	4.912	
149	0.8577	0.5801	4.935	
150	0.8411	0.6748	5.016	
151	0.8538	0.8155	4.815	
152	0.8489	0.6602	4.815	
153	0.8209	0.04773	4.103	
154	0.8536	0.5166	5.054	
155	0.8611	0.6183	4.57	
156	0.8207	0.4175	4.976	
157	0.8246	1.359	4.112	
158	0.8324	1.855	4.026	
159	0.8277	1.753	4.128	
160	0.8607	0.1201	4.002	
161	0.08571	1.828	9.986	
162	0.8327	1.579	4.098	
163	0.8592	0.2446	4.074	
164	0.4928	0.3095	9.615	
165	0.8139	0.3744	4.156	
166	0.8403	0.2262	4.213	
167	0.8505	0.1181	4.358	
168	0.8192	2.566	6.365	
169	0.8482	0.1531	4.49	
170	0.8269	0.2424	4.377	
171	0.5072	2.773	9.929	

172	0.7036	0.07444	4.599	
173	0.8542	0.2333	4.511	
174	0.8272	1.221	4.196	
175	0.8304	1.463	4.0	
176	0.8363	0.0	4.305	
177	0.854	0.3904	4.492	
178	0.8248	2.494	5.961	
179	0.8291	2.386	5.823	
180	0.7426	0.6892	5.878	
181	0.8358	0.5224	4.392	
182	0.7593	2.438	5.687	
183	0.8257	2.676	5.914	
184	0.8479	0.3557	4.0	
185	0.8146	2.692	5.753	
186	0.8094	0.4049	5.159	
187	0.8089	2.263	4.863	
188	0.8332	1.148	4.063	
189	0.819	1.031	4.534	
190	0.8227	1.561	4.262	
191	0.7782	1.096	5.04	
192	0.8187	1.462	4.421	
193	0.8351	0.5419	4.247	
194	0.8289	2.78	5.375	
195	0.5074	2.141	8.886	
196	0.8393	2.712	5.525	
197	0.8175	2.832	5.626	
198	0.8188	1.133	4.353	
199	0.829	1.064	4.201	
200	0.8371	0.9536	4.384	
201	0.8441	2.109	5.129	
202	0.7778	1.969	5.165	
203	0.8401	2.272	5.094	
204	0.8178	2.599	6.456	
205	0.8586	2.517	5.077	
206	0.8313	2.407	4.998	
207	0.8263	2.409	5.161	
208	0.8386	2.579	4.978	
209	0.6627	0.2331	8.476	
210	0.8196	0.9115	4.246	
211	0.8564	0.9852	4.077	
212	0.8582	2.475	4.847	
213	0.8564	0.8735	4.001	

214	0.8453	2.388	4.738	
215	0.8257	0.8567	4.125	
216	0.1122	1.937	9.675	
217	0.826	2.309	5.988	
218	0.8146	2.222	5.841	
219	0.8194	2.393	6.142	
220	0.8227	2.579	6.118	
221	0.8141	2.748	6.086	
222	0.7953	0.8692	5.185	
223	0.8239	2.706	6.269	
224	0.8316	1.731	4.309	
225	0.815	1.639	4.44	
226	0.7746	2.844	5.865	
227	0.8257	1.892	4.265	
228	0.8525	2.41	4.283	
229	0.7962	1.733	4.812	
230	0.8536	2.512	4.392	
231	0.8037	2.372	4.426	
232	0.8514	2.253	4.304	
233	0.8467	2.075	4.301	
234	0.8172	2.168	4.411	
235	0.8171	1.836	4.421	
236	0.8422	0.7862	4.805	
237	0.8152	2.777	6.43	
238	0.8298	2.183	6.108	
239	0.7794	2.27	6.254	
240	0.8376	1.9	4.072	
241	0.85	2.513	4.719	
242	0.8296	2.109	5.129	
243	0.8705	2.734	4.071	
244	0.8684	2.01	4.002	
245	0.849	0.273	4.776	
246	0.8482	2.872	4.749	
247	0.8404	2.114	5.12	
248	0.8621	0.9847	4.068	
249	0.8752	2.352	4.034	
250	0.5851	2.863	9.577	
251	0.8694	2.267	4.005	
252	0.8683	2.45	4.004	
253	0.8419	2.098	5.116	
254	0.8645	2.812	4.059	
255	0.8706	2.005	4.01	

256	0.858	0.8819	4.557	
257	0.8698	2.824	4.063	
258	0.799	0.5006	6.842	
259	0.8641	2.632	4.004	
260	0.8512	0.9732	4.069	
261	0.8592	2.265	4.021	
262	0.8618	2.172	4.059	
263	0.8643	2.058	4.035	
264	0.8606	0.8722	4.565	
265	0.8763	2.354	4.024	
266	0.8413	2.104	5.117	
267	0.7834	2.705	7.183	
268	0.8682	2.531	4.048	
269	0.8734	2.789	4.009	
270	0.8585	2.112	4.857	
271	0.844	0.9961	4.073	
272	0.8594	0.863	4.56	
273	0.836	2.153	5.109	
274	0.8669	2.04	4.048	
275	0.8665	2.792	4.019	
276	0.8173	0.03716	6.064	
277	0.8649	2.363	4.04	
278	0.8631	2.034	4.037	
279	0.8253	2.234	6.282	
280	0.863	2.159	4.05	
281	0.8642	2.828	4.076	
282	0.8769	2.375	4.032	
283	0.858	0.8601	4.545	
284	0.8516	2.217	4.372	
285	0.8072	2.125	4.86	
286	0.8655	2.103	4.848	
287	0.8666	2.169	4.038	
288	0.8621	2.879	4.477	
289	0.8619	2.54	4.059	
290	0.8647	0.7893	4.561	
291	0.8563	0.8007	4.811	
292	0.862	2.386	4.027	
293	0.8499	2.449	4.015	
294	0.8636	2.375	4.043	
295	0.8777	2.816	4.083	
296	0.8678	2.723	4.059	
297	0.8651	2.823	4.047	

298	0.6734	2.917	8.269	
299	0.8675	2.986	4.001	
300	0.8717	2.807	4.073	
301	0.8697	2.054	4.048	
302	0.8562	0.78	4.569	
303	0.524	0.3537	9.377	
304	0.8781	2.374	4.015	
305	0.8617	2.874	4.838	
306	0.7857	0.2558	7.351	
307	0.7192	0.004723	8.044	
308	0.872	2.804	4.043	
309	0.8655	2.383	4.006	
310	0.8757	2.813	4.028	
311	0.8686	2.79	4.051	
312	0.8731	2.778	4.017	
313	0.8567	2.721	4.072	
314	0.5644	1.887	6.597	
315	0.201	1.227	9.041	
316	0.8503	0.8109	4.797	
317	0.8454	0.08283	4.515	
318	0.8711	2.185	4.037	
319	0.2767	1.327	8.074	
320	0.7897	1.866	5.073	
321	0.8677	2.778	4.039	
322	0.8622	2.861	4.835	
323	0.8672	2.623	4.017	
324	0.8719	2.986	4.016	
325	0.1498	1.733	9.191	
326	0.8279	1.43	4.374	
327	0.8662	2.178	4.019	
328	0.8635	2.023	4.019	
329	0.4	1.269	7.527	
330	0.8633	2.639	4.027	
331	0.868	2.792	4.036	
332	0.5832	0.251	8.95	
333	0.8704	2.84	4.062	
334	0.8715	2.019	4.047	
335	0.8676	2.854	4.042	
336	0.8672	2.84	4.036	
337	0.8644	2.048	4.066	
338	0.8662	2.795	4.085	
339	0.8617	2.859	4.06	

340	0.8337	2.0	4.041	
341	0.8743	2.826	4.022	
342	0.8629	2.786	4.074	
343	0.7088	0.3358	8.313	
344	0.865	2.2	4.026	
345	0.7447	0.5153	7.721	
346	0.6763	1.053	6.099	
347	0.8704	2.196	4.049	
348	0.8668	2.85	4.077	
349	0.7331	0.8089	7.813	
350	0.3309	0.2175	9.716	
351	0.8711	2.578	4.571	
352	0.875	2.029	4.06	
353	0.8725	2.006	4.067	
354	0.8664	2.025	4.075	
355	0.8718	2.561	4.57	
356	0.8664	2.762	4.051	
357	0.8055	1.702	4.791	
358	0.8566	2.76	4.029	
359	0.7352	2.837	4.628	
360	0.8536	2.042	4.01	
361	0.4917	0.2312	9.899	
362	0.7294	2.573	4.589	
363	0.8694	2.967	4.039	
364	0.8662	2.862	4.025	
365	0.0776	1.048	9.966	
366	0.7821	0.9045	7.232	
367	0.8201	2.119	6.547	
368	0.8692	2.962	4.013	
369	0.868	2.945	4.035	
370	0.8709	2.845	4.017	
371	0.8618	2.757	4.082	
372	0.803	2.544	6.245	
373	0.8622	2.62	4.036	
374	0.8577	2.747	4.043	
375	0.5899	2.927	9.02	
376	0.8527	2.734	4.093	
377	0.8459	2.829	4.095	
378	0.8715	2.972	4.024	
379	0.8538	0.7887	4.532	
380	0.8634	2.572	4.555	
381	0.8618	2.985	4.039	

382	0.8568	0.7734	4.519	
383	0.7583	0.474	7.519	
384	0.8562	2.718	4.21	
385	0.8591	2.823	4.001	
386	0.8672	2.355	4.006	
387	0.8479	2.781	4.096	
388	0.8612	2.192	4.069	
389	0.8675	2.215	4.055	
390	0.8654	2.635	4.053	
391	0.6827	0.9022	7.686	
392	0.869	2.608	4.067	
393	0.8634	2.622	4.079	
394	0.636	0.7874	8.765	
395	0.7523	2.834	7.814	
396	0.8666	2.212	4.079	
397	0.821	0.3442	6.553	
398	0.8655	2.656	4.041	
399	0.814	2.605	6.458	
400	0.8686	2.938	4.014	
401	0.8691	2.229	4.066	
402	0.8637	2.601	4.048	
403	0.8768	2.727	4.037	
404	0.5951	2.133	9.108	
405	0.3083	1.719	8.055	
406	0.6826	2.012	8.533	
407	0.8354	0.02739	4.366	
408	0.4574	0.2615	9.928	
409	0.826	2.203	6.548	
410	0.8699	2.662	4.022	
411	0.867	2.554	4.55	
412	0.5367	1.149	6.682	
413	0.8654	2.586	4.062	
414	0.8675	2.679	4.037	
415	0.8659	2.792	4.036	
416	0.8657	2.008	4.086	
417	0.8754	2.687	4.022	
418	0.8568	2.707	4.047	
419	0.8599	2.233	4.042	
420	0.865	2.661	4.004	
421	0.8686	2.715	4.015	
422	0.8635	0.762	4.54	
423	0.8675	2.225	4.026	

424	0.8761	2.541	4.566	
425	0.8686	2.247	4.027	
426	0.8448	2.232	4.088	
427	0.878	2.251	4.065	
428	0.8721	2.231	4.007	
429	0.5048	2.561	9.771	
430	0.8619	2.682	4.058	
431	0.845	2.196	4.096	
432	0.8761	2.257	4.05	
433	0.8662	2.598	4.024	
434	0.8633	2.573	4.043	
435	0.8391	2.757	4.107	
436	0.3638	1.328	7.465	
437	0.8243	2.261	5.236	
438	0.8745	2.699	4.03	
439	0.8707	2.553	4.041	
440	0.8666	2.268	4.069	
441	0.496	0.9801	9.515	
442	0.6327	1.173	4.485	
443	0.8676	2.696	4.002	
444	0.8448	2.856	4.103	
445	0.1597	1.027	9.388	
446	0.86	2.276	4.044	
447	0.8648	2.757	4.003	
448	0.8595	2.594	4.559	
449	0.8721	2.736	4.013	
450	0.8701	2.528	4.549	
451	0.6111	0.3014	8.865	
452	0.8707	2.599	4.002	
453	0.8643	2.565	4.058	
454	0.8376	1.984	4.082	
455	0.859	2.578	4.015	
456	0.8591	0.7452	4.525	
457	0.8619	2.654	4.067	
458	0.876	2.294	4.018	
459	0.7928	2.419	7.292	
460	0.8655	2.116	4.826	
461	0.8639	2.315	4.019	
462	0.8706	2.518	4.565	
463	0.8591	2.094	4.829	
464	0.8729	2.299	4.049	
465	0.8351	1.077	4.241	

466	0.8708	2.319	4.043	
467	0.8668	2.925	4.03	
468	0.8701	2.316	4.064	
469	0.8758	2.342	4.062	
470	0.8731	2.258	4.085	
471	0.8713	2.911	4.009	
472	0.8654	2.322	4.084	
473	0.8687	2.301	4.083	
474	0.8369	2.312	4.102	
475	0.8726	2.189	4.002	
476	0.8331	2.348	4.089	
477	0.8559	2.872	4.089	
478	0.8673	2.89	4.022	
479	0.7251	2.531	4.59	
480	0.8728	2.915	4.054	
481	0.8727	2.881	4.067	
482	0.8407	2.275	4.091	
483	0.8598	2.906	4.039	
484	0.8744	2.332	4.001	
485	0.873	2.912	4.077	
486	0.8545	0.7402	4.549	
487	0.8659	2.543	4.02	
488	0.8577	2.139	4.83	
489	0.225	0.7789	9.57	
490	0.8496	2.933	4.058	
491	0.8663	2.977	4.062	
492	0.8589	2.877	4.003	
493	0.8424	2.7	4.091	
494	0.8632	2.703	4.227	
495	0.6925	0.8866	8.079	
496	0.8681	2.291	4.068	
497	0.2973	1.818	8.038	
498	0.851	2.895	4.107	
499	0.8645	2.897	4.083	
500	0.8436	0.9324	4.803	
501	0.8597	2.128	4.802	
502	0.8724	2.495	4.554	
503	0.8766	2.365	4.067	
504	0.8707	2.954	4.077	
505	0.8588	2.099	4.8	
506	0.851	2.725	4.233	
507	0.8584	2.06	4.805	

508	0.8359	2.93	4.099	
509	0.8353	2.075	5.118	
510	0.8051	1.661	4.673	
511	0.2483	1.56	8.292	
512	0.8503	2.052	4.777	
513	0.868	2.503	4.531	
514	0.8493	2.082	4.78	
515	0.8516	0.9062	4.805	
516	0.8714	2.525	4.524	
517	0.8685	2.516	4.016	
518	0.8618	2.152	4.808	
519	0.8177	1.894	4.352	
520	0.871	2.481	4.539	
521	0.8354	2.803	4.109	
522	0.8673	2.304	4.001	
523	0.864	2.463	4.564	
524	0.8688	2.678	4.084	
525	0.8385	2.656	4.103	
526	0.874	2.497	4.039	
527	0.862	2.479	4.01	
528	0.8705	2.21	4.001	
529	0.8598	2.472	4.032	
530	0.8425	2.973	4.097	
531	0.8734	2.498	4.06	
532	0.8519	2.503	4.509	
533	0.87	2.165	4.0	
534	0.8525	2.396	4.22	
535	0.7824	2.485	6.822	
536	0.865	2.481	4.055	
537	0.8661	2.469	4.516	
538	0.7182	0.3037	7.819	
539	0.8733	2.446	4.535	
540	0.5094	0.454	9.253	
541	0.8518	2.147	4.784	
542	0.8072	0.2699	5.168	
543	0.8666	2.567	4.084	
544	0.8616	2.438	4.561	
545	0.8507	2.449	4.501	
546	0.8704	2.486	4.58	
547	0.4217	2.966	8.981	
548	0.8626	2.045	4.825	
549	0.8566	2.68	4.207	

550	0.8713	2.473	4.082	
551	0.6252	1.032	6.146	
552	0.7409	0.3682	7.651	
553	0.8663	2.42	4.534	
554	0.868	2.455	4.067	
555	0.866	2.544	4.087	
556	0.8679	2.145	4.016	
557	0.8591	2.494	4.083	
558	0.8402	2.548	4.111	
559	0.8558	2.069	4.841	
560	0.8676	2.52	4.084	
561	0.8446	2.467	4.107	
562	0.866	2.396	4.068	
563	0.5117	0.9022	9.587	
564	0.8509	2.435	4.517	
565	0.8528	2.47	4.486	
566	0.8465	2.677	4.248	
567	0.8446	2.578	4.112	
568	0.8672	2.417	4.057	
569	0.8642	2.436	4.042	
570	0.8416	2.698	4.268	
571	0.8119	2.084	4.866	
572	0.8649	2.136	4.038	
573	0.7481	2.463	4.586	
574	0.8437	2.513	4.113	
575	0.8639	2.412	4.01	
576	0.8501	2.742	4.259	
577	0.8691	2.42	4.035	
578	0.8598	2.547	4.528	
579	0.8713	2.379	4.086	
580	0.8427	2.412	4.096	
581	0.8597	2.64	4.201	
582	0.8473	2.385	4.111	
583	0.8706	2.405	4.554	
584	0.8492	2.635	4.168	
585	0.849	0.1421	4.015	
586	0.8525	0.006214	4.796	
587	0.8668	2.119	4.023	
588	0.8659	2.4	4.53	
589	0.8126	2.887	4.454	
590	0.8701	2.087	4.019	
591	0.8641	2.086	4.054	

592	0.8386	2.657	4.187	
593	0.8514	2.532	4.497	
594	0.8684	2.377	4.54	
595	0.846	0.07121	4.776	
596	0.8598	2.11	4.781	
597	0.8606	2.168	4.829	
598	0.8648	2.071	4.082	
599	0.8464	2.376	4.512	
600	0.8641	2.122	4.058	
601	0.8393	2.055	4.1	
602	0.8615	2.18	4.809	
603	0.8678	2.393	4.58	
604	0.8595	0.1169	4.029	
605	0.8593	2.373	4.57	
606	0.8619	0.0975	4.009	
607	0.8615	2.204	4.829	
608	0.8708	2.107	4.036	
609	0.8663	2.071	4.001	
610	0.8616	2.181	4.851	
611	0.8633	2.348	4.544	
612	0.8656	2.108	4.003	
613	0.8225	0.7066	6.219	
614	0.834	2.692	4.301	
615	0.8459	2.681	4.701	
616	0.858	2.645	4.23	
617	0.8737	2.419	4.577	
618	0.8624	2.105	4.074	
619	0.8492	2.643	4.254	
620	0.8716	2.138	4.074	
621	0.749	2.412	4.6	
622	0.8142	1.293	4.662	
623	0.8548	2.342	4.512	
624	0.8551	2.412	4.246	
625	0.1893	1.031	8.886	
626	0.5778	0.007506	8.785	
627	0.8538	2.881	4.506	
628	0.8524	2.854	4.491	
629	0.8465	0.2746	4.927	
630	0.8363	2.122	4.093	
631	0.8462	2.168	4.09	
632	0.8474	2.436	4.26	
633	0.8459	2.386	4.252	

634	0.6067	0.199	8.558	
635	0.8428	2.17	4.784	
636	0.355	1.912	7.813	
637	0.7061	0.3453	7.927	
638	0.8596	2.424	4.218	
639	0.8522	2.021	4.109	
640	0.8605	2.017	4.83	
641	0.8564	0.6802	4.547	
642	0.8046	2.203	4.863	
643	0.3	1.703	7.403	
644	0.3968	2.878	8.72	
645	0.8603	2.04	4.847	
646	0.8505	2.213	4.806	
647	0.8611	2.91	4.494	
648	0.823	2.037	4.877	
649	0.2957	0.0857	8.805	
650	0.8609	0.8273	4.552	
651	0.8521	0.6987	4.573	
652	0.8273	2.69	6.116	
653	0.857	0.08283	4.032	
654	0.8477	2.62	4.225	
655	0.8686	2.902	4.531	
656	0.8546	2.925	4.519	
657	0.8551	0.8334	4.579	
658	0.8607	0.6549	4.572	
659	0.8653	2.337	4.577	
660	0.8688	2.316	4.548	
661	0.2563	1.009	8.698	
662	0.139	1.745	9.635	
663	0.8736	2.931	4.551	
664	0.8573	0.6483	4.535	
665	0.8658	2.318	4.526	
666	0.8721	2.905	4.569	
667	0.4754	1.422	7.285	
668	0.7375	2.308	4.586	
669	0.8646	2.957	4.562	
670	0.8679	2.291	4.531	
671	0.8116	0.2469	4.115	
672	0.8759	2.937	4.581	
673	0.8704	2.876	4.551	
674	0.6706	0.6746	4.59	
675	0.8567	2.941	4.483	

676	0.8688	2.991	4.553	
677	0.729	2.886	4.595	
678	0.8554	0.51	4.824	
679	0.8639	2.234	4.827	
680	0.8608	0.8323	4.519	
681	0.8697	2.98	4.58	
682	0.8691	2.854	4.538	
683	0.8457	2.41	4.192	
684	0.5981	0.3917	9.476	
685	0.6898	1.42	5.844	
686	0.7948	2.33	5.276	
687	0.8316	2.069	6.383	
688	0.8612	0.6761	4.521	
689	0.2512	1.14	8.247	
690	0.8662	2.946	4.531	
691	0.8531	2.914	4.466	
692	0.8454	0.03026	4.778	
693	0.8575	2.29	4.506	
694	0.7376	2.356	4.599	
695	0.7165	0.807	4.587	
696	0.8643	2.317	4.495	
697	0.8523	0.5047	4.798	